Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Resumen 2do Parcial

Ignacio Sammartino

Índice

Clase 7 . . . 3
Protocolo CAN (Controller Area Network) . . . 3
Arquitectura y Características Principales . . . 3
Capa Física y Señal Diferencial . . . 3
Estructura de Trama, Arbitraje y Bit Stuffing . . . 4
Base de Tiempo y Sincronización (Nominal Bit Time) . . . 5
Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time) . . . 5
Filtrado por Hardware y Manejo de Errores . . . 7
Clase 8 . . . 8
Periféricos de Temporización y Módulo FlexTimer (FTM) . . . 8
Fundamentos y Necesidad de Hardware Dedicado . . . 8
Arquitectura del Módulo FlexTimer (FTM) en Kinetis K64F . . . 10
Modos de Operación y Configuración por Canal . . . 13
Fuentes de Interrupción y Vectores NVIC . . . 15
Clase 9 . . . 16
ADC: Conversor Analógico–Digital . . . 16
Arquitectura y Señales de Entrada . . . 16
Selección de Canal e Inicio de Conversión . . . 16
Reloj del ADC . . . 17
Resolución del ADC . . . 17
Lectura del Resultado y Flag EOC . . . 17
Comparador Analógico (CMP) . . . 18
PIT: Periodic Interrupt Timer . . . 20

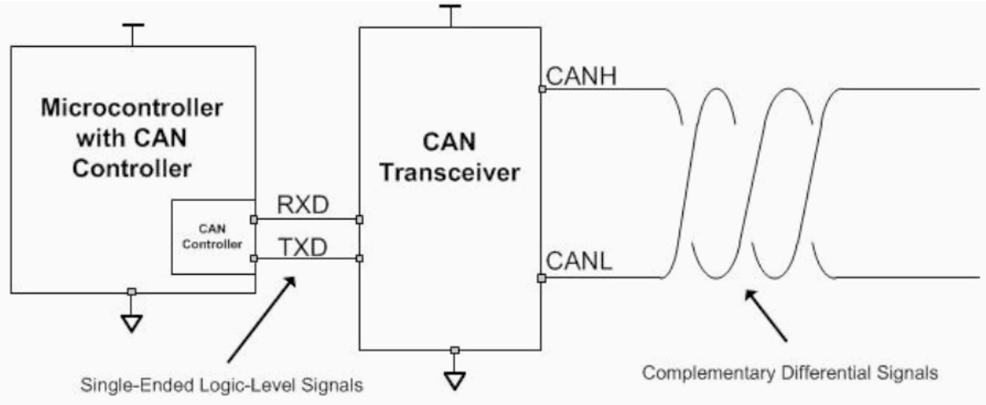
- Descripción General y Uso . . . 20
- Clock y Habilitación . . . 20
- Registros Principales . . . 20
- Modo Chain (Timers Encadenados) . . . 20
- PIT como Trigger para DMA y ADC . . . 21
- DMA: Direct Memory Access . . . 22
- Concepto y Motivación . . . 22
- Arquitectura del eDMA en el K64F . . . 22
- DMAMUX: Modos de Operación . . . 23
- Tipos de Fuentes DMA . . . 24
- TCD: Transfer Control Descriptor . . . 25
- Modos Avanzados del DMA . . . 29

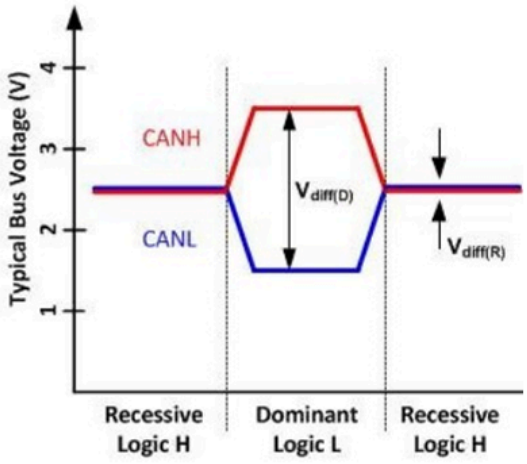
Clase 10 . . . 30

- Introducción a ARM Cortex-M4: Startup y Toolchain . . . 30
- CPU vs. microcontrolador . . . 30
- Qué es ARM y qué define una arquitectura . . . 30
- Mapa de registros del Cortex-M4 . . . 32
- Arquitectura del Cortex-M4 y mapa de memoria . . . 33
- ¿Dónde se guarda cada variable? . . . 35
- Booteo del ARM Cortex-M4 . . . 37
- La toolchain ARM . . . 38
- Secciones y linker script . . . 40
- Código de startup . . . 43
- Startup real del Kinetis (`startup_mk64f12.c`) . . . 44
- Resumen final . . . 46

Clase 7

Protocolo CAN (Controller Area Network)





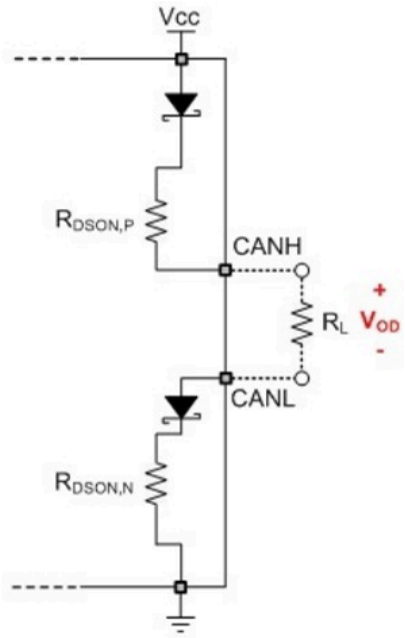


Figure 2: CAN bus signal levels

Arquitectura y Características Principales

CAN es un estándar de bus serie asíncrono, multimaestro y altamente robusto diseñado originalmente por Bosch para aplicaciones automotrices e industriales (ISO 11898). Permite que múltiples Unidades de Control Electrónico (ECUs) se comuniquen sobre un bus común sin necesidad de un nodo maestro central o controlador host.

Características fundamentales del protocolo:

- **Multimaestro / Asíncrono:** Cualquier nodo puede iniciar la transmisión cuando el bus está libre (**Idle**).
- **Orientado a Mensajes:** Los mensajes no contienen direcciones de nodo origen o destino; en su lugar, incorporan un **Identificador (ID)** que define la naturaleza del dato y su prioridad.
- **Inmunidad al Ruido:** Utiliza señalización diferencial sobre par trenzado.
- **Arbitraje No Destructivo:** Resuelve colisiones en tiempo real mediante lógica de bus dominante/recesivo (**Wired-AND**).
- **Detección y Manejo de Errores:** Lógica integrada por hardware para contención y aislamiento de fallas.

Capa Física y Señal Diferencial

A nivel eléctrico, el bus CAN se compone de un par de cables trenzados (**twisted pair**) denominados **CANH** ($\mathrm{CAN}+$ y **CANL** ($\mathrm{CAN}-$, acompañados por una referencia común de masa (GND). Para evitar reflexiones de

onda electromagnética en altas frecuencias, el bus exige **resistencias de terminación de** 120Ω instaladas en ambos extremos físicos de la línea.

El bus opera bajo dos estados eléctricos distintos que implementan la lógica **Wired-AND**:

⚙️ Estado Recesivo ('1' Lógico)

- Transceptores en alta impedancia (apagados).
- Las líneas descansan pasivamente a través de resistencias **pull-middle**.
- Voltaje: $V_{\text{CANH}} = 2.5\ \text{V}$, $V_{\text{CANL}} = 2.5\ \text{V}$.
- Tensión diferencial: $V_{\text{diff}} = V_{\text{CANH}} - V_{\text{CANL}} = 0\ \text{V}$.

⚙️ Estado Dominante ('0' Lógico)

- Transceptores activos excitando la línea.
- Se fuerzan niveles de tensión separados.
- Voltaje: $V_{\text{CANH}} = 3.5\ \text{V}$, $V_{\text{CANL}} = 1.5\ \text{V}$.
- Tensión diferencial: $V_{\text{diff}} = 2.0\ \text{V}$.

💡 Lógica Wired-AND y Colisión

Al ser un bus cableado en configuración **Wired-AND**, si dos nodos intentan transmitir simultáneamente y uno de ellos impone un estado **Dominante ('0')** mientras el otro envía un **Recesivo ('1')**, el estado dominante sobrescribe físicamente al recesivo en todo el bus. Esto permite resolver colisiones sin destruir el mensaje dominante.

Estructura de Trama, Arbitraje y Bit Stuffing

Trama de Datos Estándar (Standard Data Frame)
Un **Data Frame** Estándar (CAN 2.0A) se compone de los siguientes campos secuenciales:

Campo	Longitud	Descripción
SOF (Start of Frame)	1 bit	Bit dominante ($1 \rightarrow 0$) que marca el inicio de trama y sincroniza a todos los nodos.
Arbitration Field	12 bits	Contiene el **Identificador (ID)** de 11 bits y el bit **RTR** (0 = Data Frame, 1 = Remote Frame).
Control Field	6 bits	Incluye bits de extensión/reserva (`IDE`, `r0`) y el `DLC` (Data Length Code, 4 bits: $0..8$ bytes).
Data Field	$0..8$ bytes	Carga útil (**payload**) del mensaje ($0..64$ bits).
CRC Field	16 bits	Secuencia de verificación de redundancia cíclica de 15 bits + 1 bit `CRC Delimiter` (recesivo).
ACK Field	2 bits	`ACK Slot` (1 bit, enviado recesivo por el emisor y forzado a dominante por receptores con CRC válido) + `ACK Delimiter` (1 bit recesivo).
EOF (End of Frame)	7 bits	Fin de trama compuesto por 7 bits recesivos consecutivos.
ITM (Intermission)	3 bits	Espacio de descanso recesivo obligatorio entre tramas (**Interframe Space**).

Arbitraje No Destructivo (CSMA/CD + AMP)
Cuando el bus entra en estado pasivo (**Idle**), cualquier nodo con un mensaje pendiente puede emitir el bit `SOF`. Si dos o más nodos inician la transmisión en el mismo instante:

1. Cada nodo transmite los bits de su `ID` mientras monitorea continuamente el nivel físico del bus.
2. Si un nodo envía un bit **recesivo ('1')** en su ID pero lee un nivel **dominante ('0')** en el bus, detecta que otro mensaje en competencia tiene mayor prioridad.
3. El nodo perdedor retira su transmisor inmediatamente, pasa a modo de solo escucha (**listening only**) y aguarda a que el bus vuelva a quedar libre.

4. **Regla de prioridad:** Cuanto **menor sea el valor numérico del ID**, mayor es su prioridad en la red (el bit dominante '0' predomina en las primeras posiciones).

Regla de Bit Stuffing

Debido a que CAN es un protocolo asíncrono sin línea física de reloj, los receptores dependen de las transiciones de voltaje ($1 \rightarrow 0$) para sincronizar sus relojes internos.

> ⚠️ **Mecanismo de Bit Stuffing**
>
> Si el transmisor detecta **5 bits consecutivos de la misma polaridad** en el flujo de datos, el hardware inserta automáticamente **1 bit de relleno (Stuff Bit)** de polaridad opuesta antes de enviar el siguiente bit. Los nodos receptores remueven este bit de relleno de forma transparente antes de procesar la trama.
>
> Esta regla se aplica estrictamente desde el bit `SOF` hasta el final de la secuencia `CRC` (se excluyen el `CRC Delimiter`, el campo `ACK` y el `EOF`).

Base de Tiempo y Sincronización (Nominal Bit Time)

División del Bit en Time Quanta t_q

En redes CAN, la duración de cada bit (**Nominal Bit Time**) se subdivide en N unidades discretas de tiempo denominadas **Time Quanta** (t_q. La duración de 1 t_q se deriva del reloj interno del microcontrolador (f_{osc} mediante el divisor **Baud Rate Prescaler** (BRP.

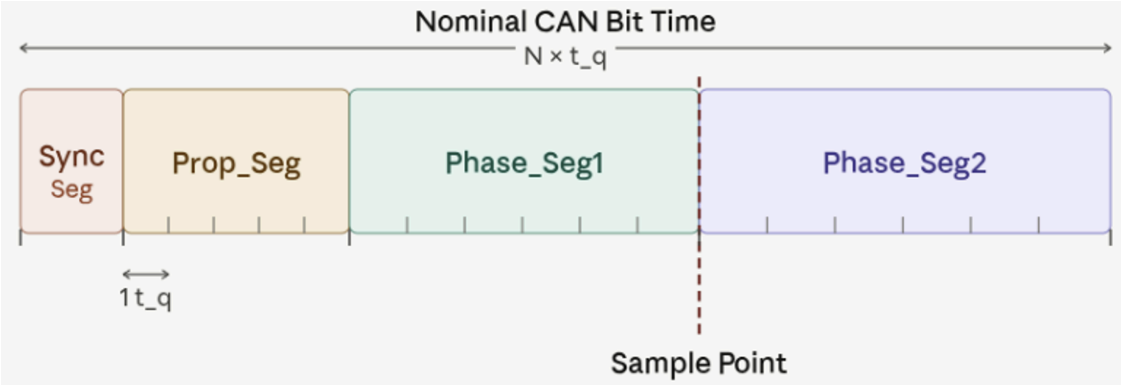


Figura 4: Estructura interna del Nominal Bit Time en sus cuatro segmentos de Time Quanta

El tiempo de bit se divide exactamente en cuatro segmentos configurables:

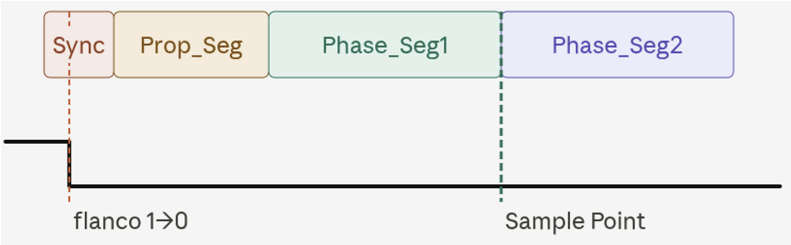
1. **Sync_Seg (Synchronization Segment):** Fijado siempre en 1 t_q. Es el intervalo donde se espera que ocurra el flanco descendente ($1 \rightarrow 0$ que marca el inicio del bit.
2. **Prop_Seg (Propagation Segment):** Configurable de 1 a 8 t_q. Compensa los retrasos físicos de propagación de las ondas electromagnéticas a lo largo del cable y los retardo internos de los transceptores.
3. **Phase_Seg1 (Phase Buffer Segment 1):** Configurable de 1 a 8 t_q. Se ubica inmediatamente antes del **Sample Point**. Se alarga en la resincronización si el flanco llega tarde.
4. **Phase_Seg2 (Phase Buffer Segment 2):** Configurable de 1 a 8 t_q. Se ubica inmediatamente después del **Sample Point**. Se acorta en la resincronización si el flanco llega temprano.

> 💡 **Sample Point (Punto de Muestreo)**
>
> El **Sample Point** es el instante exacto, situado justo al final de `Phase_Seg1`, en el que el nodo evalúa el estado lógico del bus (dominante o recesivo). Se suele configurar típicamente entre el 75% y el 87.5% de la duración total del bit.

Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time)

En el protocolo CAN, la resincronización continua de los nodos receptores se realiza exclusivamente sobre los flancos descendentes ($1 \rightarrow 0$).

Caso 1: Flanco Ideal (Sin Ajuste)



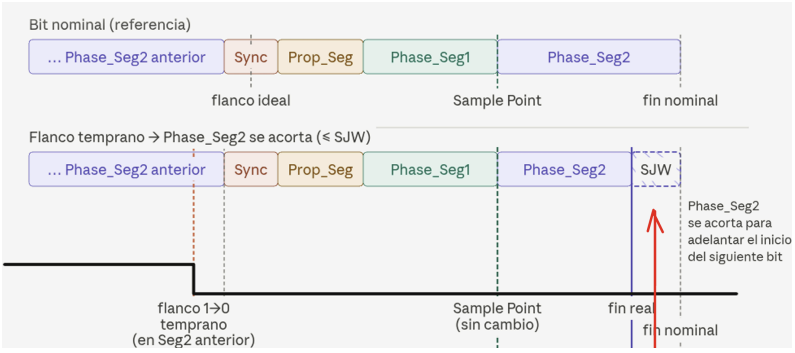
- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega exactamente dentro del segmento `Sync_Seg` (de duración fija $1t_q$).
- **Estado de Reloj:** El oscilador del nodo receptor está perfectamente sincronizado con el del transmisor.
- **Acción:** No se realiza ningún ajuste en la duración de los segmentos ($\Delta t_q = 0$) y el **Sample Point** permanece en su posición nominal.

Caso 2: Flanco Tardío (Reloj del Transmisor Más Lento)



- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega con retraso, cayendo en los segmentos `Prop_Seg` o `Phase_Seg1` en lugar de `Sync_Seg`.
- **Estado de Reloj:** El reloj del nodo transmisor es **más lento** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **alarga** el segmento `Phase_Seg1` agregando ciclos t_q adicionales.
  - La cantidad de ciclos agregados es igual al error de fase detectado, acotado por el valor máximo de `SJW` ($+\Delta t_q \leq \mathrm{SJW}$).
  - **Efecto:** Se desplaza el **Sample Point** hacia la derecha (retrasando el momento de muestreo) para capturar correctamente el dato.

Caso 3: Flanco Temprano (Reloj del Transmisor Más Rápido)



- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega antes de lo esperado, cayendo dentro del segmento `Phase_Seg2` del bit anterior.
- **Estado de Reloj:** El reloj del nodo transmisor es **más rápido** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **acorta** el segmento `Phase_Seg2` substrayendo ciclos t_q de su duración.
  - La reducción está acotada por el valor máximo configurado en `SJW` ($-\Delta t_q \leq \mathrm{SJW}$).
  - **Efecto:** Adelanta la finalización del bit actual e inicia el próximo ciclo antes de lo nominal para reengancharse con el transmisor rápido.

Caso	Ubicación del Flanco	Clock Transmisor	Ajuste de Hardware
Caso 1: Ideal	Dentro de `Sync_Seg`	Sincronizado	Sin ajuste ($\Delta t_q = 0$).
Caso 2: Tardío	En `Prop_Seg` / `Phase_Seg1`	Más lento	Se alarga `Phase_Seg1` ($+\Delta t_q \leq \text{SJW}$).
Caso 3: Temprano	En `Phase_Seg2` anterior	Más rápido	Se acorta `Phase_Seg2` ($-\Delta t_q \leq \text{SJW}$).

> ⚠️ **¡Sincronización Solo en Flancos Descendentes!**
>
> En CAN, la sincronización se realiza **exclusivamente sobre los flancos descendentes ($1 \rightarrow 0$, paso a dominante)**.
>
> Los flancos ascendentes ($0 \rightarrow 1$, paso a recesivo) **no se utilizan** debido a que la subida del bus es pasiva (depende de la resistencia **pull-middle** y capacitancias parásitas del cable), generando pendientes lentas e imprecisas.

Filtrado por Hardware y Manejo de Errores

Filtrado de Mensajes (Filter y Mask)

Para evitar sobrecargar la CPU del microcontrolador analizando mensajes no relevantes, el módulo controlador CAN (como el FlexCAN en el Kinetis K64F) filtra las tramas por hardware mediante dos registros:

- **Filter (Identificador de Filtro):** Contiene el patrón binario de ID específico que el nodo está programado para recibir.
- **Mask (Máscara de Aceptación):** Define qué bits del ID deben coincidir obligatoriamente:
  - Bit de máscara en `'1'`: **Verificación estricta** (el bit del ID entrante debe ser idéntico al bit del filtro).
  - Bit de máscara en `'0'`: **Don't Care** (se ignora la coincidencia de ese bit en particular).

Estados del Nodo y Manejo de Errores

Cada nodo CAN posee dos contadores internos de error mantenidos por hardware: `TEC` (**Transmit Error Counter**) y `REC` (**Receive Error Counter**). Según sus valores, el nodo transiciona entre tres estados de operación:

Estado del Nodo	Condición de Errores	Comportamiento en la Red
Error Active	$\text{TEC} < 128$ y $\text{REC} < 128$	Operación normal. Al detectar un error, transmite un **Active Error Flag** (6 bits dominantes), abortando la trama de toda la red.
Error Passive	$\text{TEC} \geq 128$ o $\text{REC} \geq 128$	El nodo está degradado. Transmite un **Passive Error Flag** (6 bits recesivos), evitando interrumpir la comunicación de otros nodos.
Bus Off	$\text{TEC} > 255$	El nodo se desconecta lógicamente del bus. No puede transmitir ni recibir hasta reiniciar el módulo FlexCAN.

Clase 8

Periféricos de Temporización y Módulo FlexTimer (FTM)

Fundamentos y Necesidad de Hardware Dedicado

En sistemas embebidos en tiempo real, el uso de interrupciones periódicas por software para generar o medir pulsos presenta limitaciones críticas cuando los anchos de pulso son muy angostos o cuando se requiere una precisión estricta sin sufrir la latencia de atención de la CPU.

Por esta razón, los microcontroladores incorporan bloques periféricos de **Timer por hardware**. Los dos conceptos fundamentales sobre los que se construyen estos periféricos son:

- **Output Compare (OC):** Permite programar un cambio de estado automático en un pin de salida (Toggle, Set, Clear) cuando el contador alcanza un valor predeterminado en el registro de comparación. Este evento ocurre por hardware de forma determinística, eliminando el impacto del jitter e impredecibilidad causados por la latencia de la ISR.

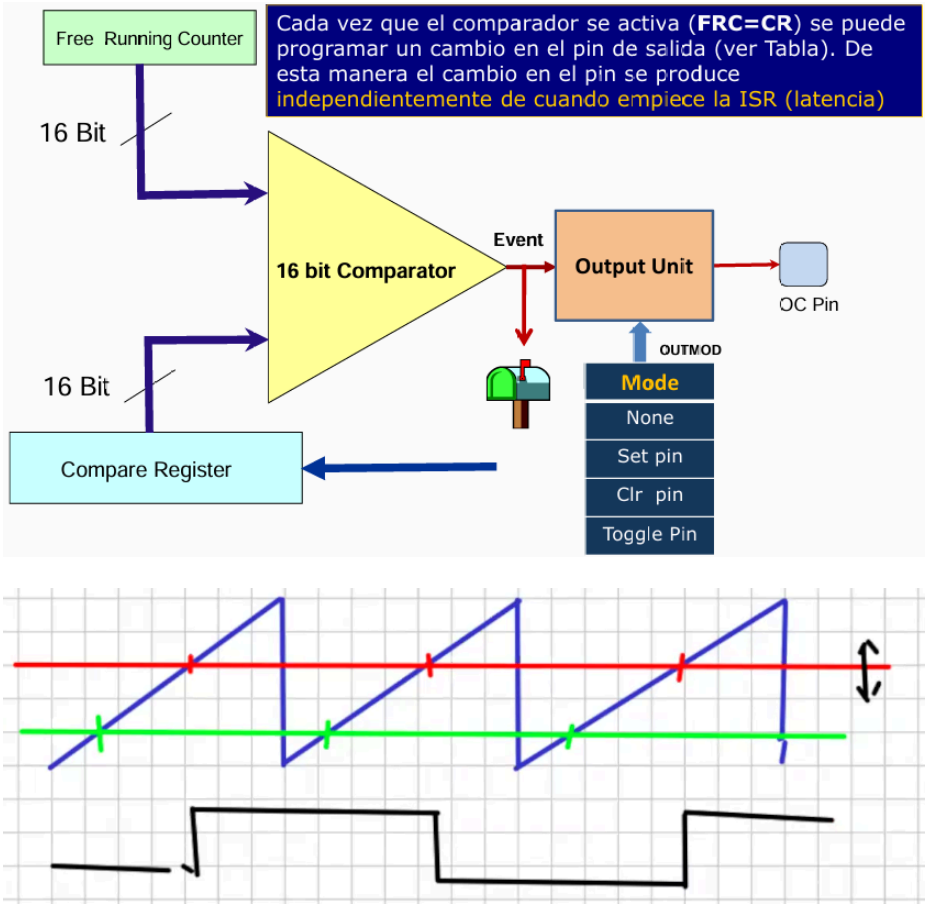


Figura 9: Ejemplo de uso con duty cycle constante

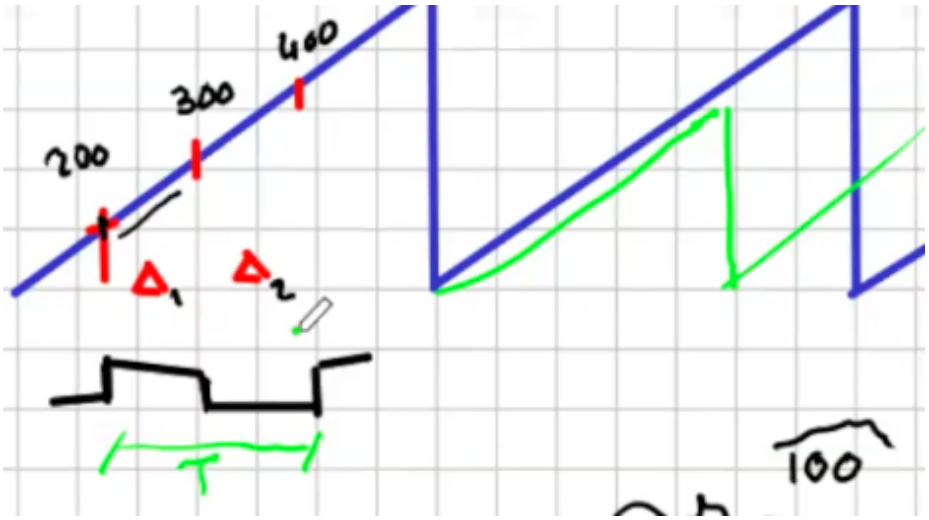


Figura 10: Ejemplo de uso con duty cycle utilizando deltas

- **Input Capture (IC):** Registra (**latch**) el valor instantáneo del contador principal ante la llegada de un flanco en un pin de entrada. Se utiliza para medir frecuencias, períodos o anchos de pulso extremadamente angostos sin sobrecargar la CPU.

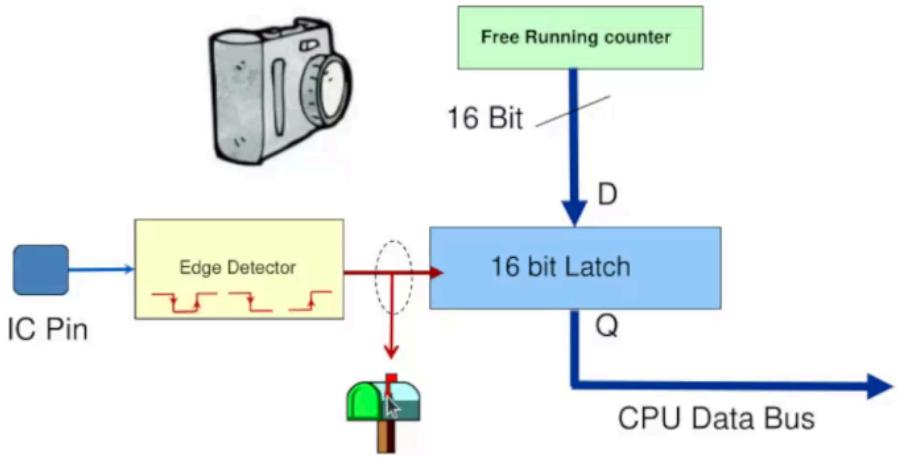


Figura 11: 'Se le saca una foto al contador en el instante del flanco de entrada'

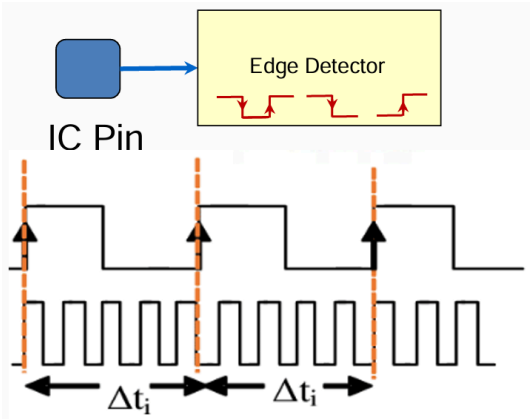


Figura 12: t_i corresponde al clock de FRC (Free Running Counter).)

> 💡 **Principio de Funcionamiento por Hardware**
>
> Ambas técnicas se basan en un contador principal de 16 bits (**Free Running Counter / FTM Counter**) accionado por un reloj del sistema [1, 11, 12]. Tanto el latch de captura como la conmutación de los pines de salida se realizan en el mismo ciclo de reloj del evento físico.

Arquitectura del Módulo FlexTimer (FTM) en Kinetis K64F

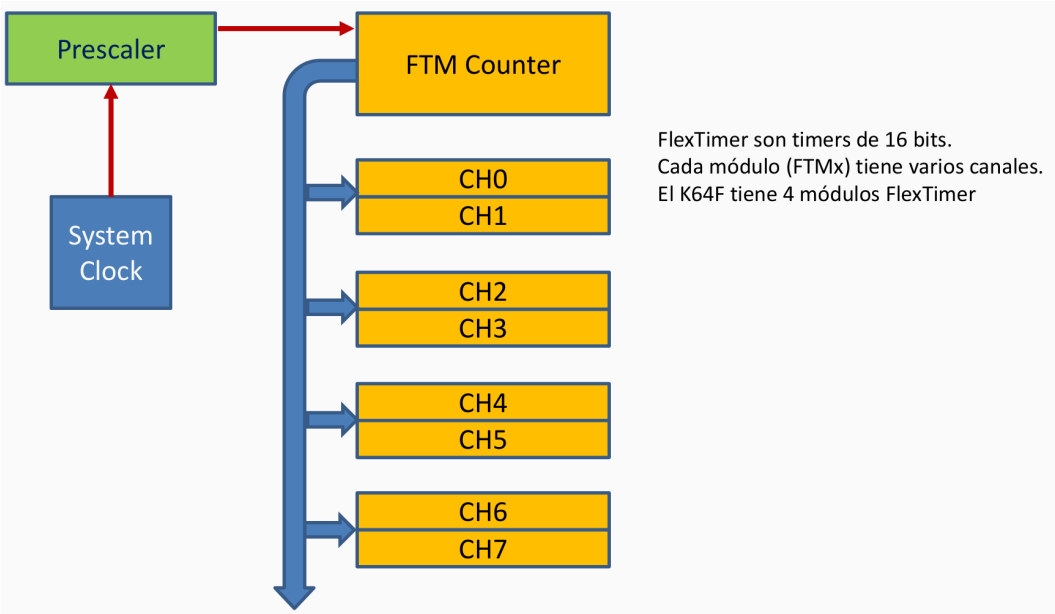


Figura 13: FTM counter y sus-submodulos.

El **FlexTimer Module (FTM)** es un temporizador multicanal de 16 bits integrado en la subfamilia Kinetis K64 (microcontrolador `MK64FN1M0VLL12` montado en la placa FRDM-K64F).

Instancias y Distribución de Canales

El microcontrolador K64F dispone de 4 instancias físicas del módulo FTM:

Instancia FTM	N° de Canales	Funciones Principales / Usos
FTM0	8 canales (CH0..CH7)	Control de motores trifásicos, PWM general, motores paso a paso.
FTM1	2 canales (CH0..CH1)	Decodificador en cuadratura (**Quadrature Decoder**) o propósito general.
FTM2	2 canales (CH0..CH1)	Decodificador en cuadratura o propósito general.
FTM3	8 canales (CH0..CH7)	Control de motores trifásicos, PWM y medición de señales.

> ⚙️ **Habilitación del Reloj (Clock Gating)**
>
> Antes de acceder a los registros de cualquier módulo FTM, es obligatorio activar la señal de reloj (**Clock Gate**) en el módulo SIM (**System Integration Module**):
> - **FTM0, FTM1, FTM2:** Se habilitan mediante los bits `SIM_SCGC6_FTM0_MASK`, `SIM_SCGC6_FTM1_MASK`, `SIM_SCGC6_FTM2_MASK` en `SIM->SCGC6`.
> - **FTM2, FTM3:** Se habilitan mediante los bits `SIM_SCGC3_FTM2_MASK` y `SIM_SCGC3_FTM3_MASK` en `SIM->SCGC3`.

Fuentes de Reloj (Clock Source) y Prescaler

El contador del FTM recibe su señal de reloj desde una llave de selección configurable en el registro de control `FTMx_SC` mediante el campo `CLKS[1:0]`:

- `00`: Sin reloj seleccionado (deshabilita el contador FTM).
- `01`: **System Clock:** Corresponde internamente al **Bus Clock** del microcontrolador ($f_{\text{bus}} = 50$ MHz en la placa FRDM-K64F a 100 MHz de Core).
- `10`: **Fixed Frequency Clock (`MCGFFCLK`):** Reloj de frecuencia fija independiente.
- `11`: **External Clock (`FTM_CLKINx`):** Reloj desde pines externos seleccionados vía `SIM_SOPT4`.

El valor del reloj seleccionado se divide posteriormente mediante un **Prescaler (PS)** de 3 bits (`PS[2:0]` en `FTMx_SC`), el cual soporta factores de división entre 1 y 128 (2^n con $n = 0..7$):

$$f_{\text{FTM_CNT}} = \frac{f_{\text{Clock_Source}}}{\text{Prescaler}}$$

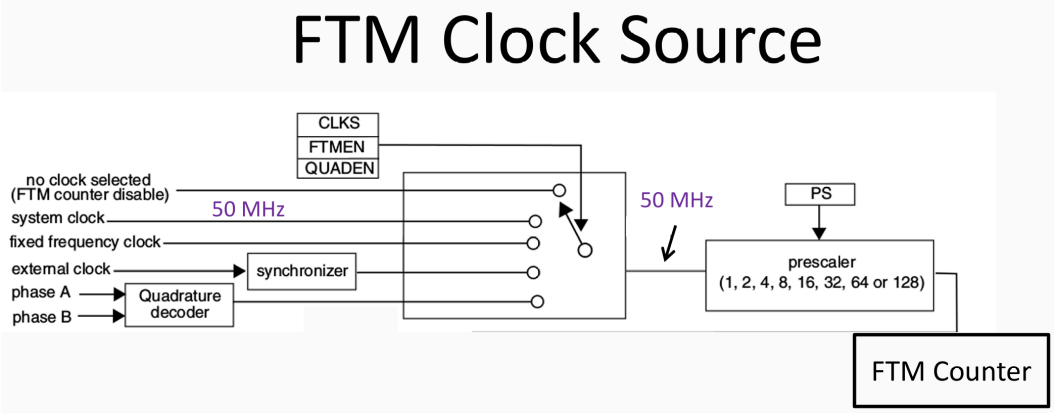


Figura 14: Clock Source

🛠️ Cálculo de Frecuencia y Resolución de Conteo

Si se selecciona el **Bus Clock** (50 MHz) y se configura un prescaler de 4 (`FTM_PSC_x4`): $f_{\text{FTM_CNT}} = \frac{50\text{ MHz}}{4} = 12.5\text{ MHz}$ Resolución de conteo (período de tick) $= \frac{1}{12.5\text{ MHz}} = 80\text{ ns}$

```
void FTM_SetPrescaler (FTM_t ftm, FTM_Prescal_t data)
{
    ftm->SC = (ftm->SC & ~FTM_SC_PS_MASK) | FTM_SC_PS(data);
}
```

En MK64F12.h

```
#define FTM_SC_PS(x)        (((uint32_t)(((uint32_t)(x))<<FTM_SC_PS_SHIFT))&FTM_SC_PS_MASK)
#define FTM_SC_PS_MASK      0x7u
#define FTM_SC_PS_SHIFT     0
```

```
typedef enum
{
FTM_PSC_x1= 0x00,
FTM_PSC_x2= 0x01,
FTM_PSC_x4= 0x02,
FTM_PSC_x8= 0x03,
FTM_PSC_x16= 0x04,
FTM_PSC_x32= 0x05,
FTM_PSC_x64= 0x06,
FTM_PSC_x128= 0x07,
} FTM_Prescal_t;
```

Ejemplo:

```
FTM_SetPrescaler(FTM0, FTM_PSC_x4); // BusClock/2=Sysclk/2= 50MHz y presccaler = divx4
```

Entonces: FTM_Coutner_Clk= 50MHz /4= 12.5MHz (Resolución 80 ns)

Modos de Conteo del Contador (`FTM Counter`)

El FTM soporta tres modos de conteo configurables mediante los registros `FTMx_SC` y `FTMx_QDCTRL`:

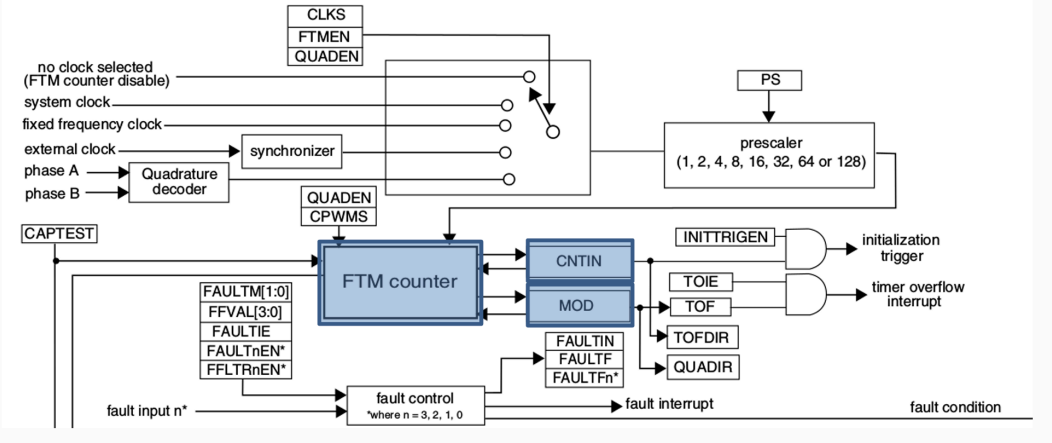


Figura 16: FTM: Up Counting

1. **Up Counting (Conteo Ascendente):**
   - **Configuración:** `QUADEN = 0`, `CPWMS = 0`.
   - **Comportamiento:** El contador se carga con `CNTIN` y se incrementa en cada tick hasta alcanzar `MOD`. Al llegar a `MOD`, se recarga automáticamente con `CNTIN` e impone `TOF = 1` (Timer Overflow Flag).
   - **Período de conteo:**

$$\text{Período} = (\text{MOD} - \text{CNTIN} + 1) \times t_{\text{FTM_CNT}}$$

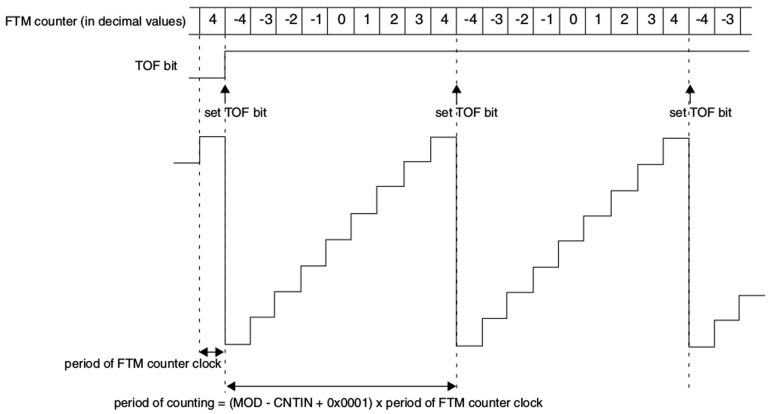


Figura 17: Up Counting, cuenta la cantidad de overflows.

2. **Up-Down Counting (Conteo Ascendente/Descendente - Center Aligned):**
   - **Configuración:** `QUADEN = 0`, `CPWMS = 1`.
   - **Comportamiento:** Cuenta desde `CNTIN` hasta `MOD`, y al llegar a `MOD` comienza a decrementar hasta regresar a `CNTIN`. El flag `TOF = 1` se activa al pasar de `MOD` a `MOD - 1`.
   - **Período de conteo:**

$$\text{Período} = 2 \times (\text{MOD} - \text{CNTIN}) \times t_{\text{FTM_CNT}}$$

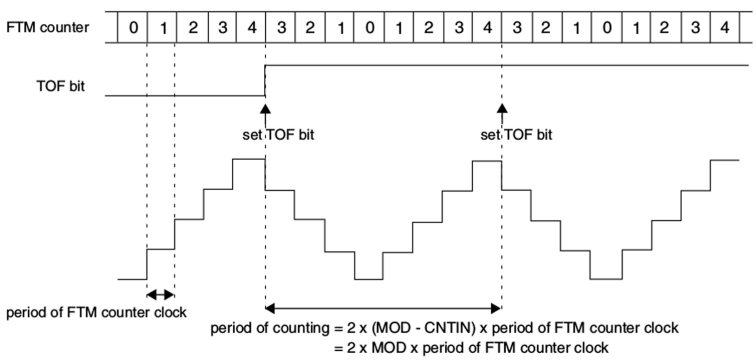


Figura 18: Up-Down Counting

3. **Free Running Counter:**

- **Comportamiento:** Se obtiene fijando `CNTIN = 0x0000` y `MOD = 0xFFFF`. El contador recorre todo el rango de 16 bits ($0..65535$) continuamente. `TOF = 1` cuando el contador pasa de 0xFFFF to 0x0000

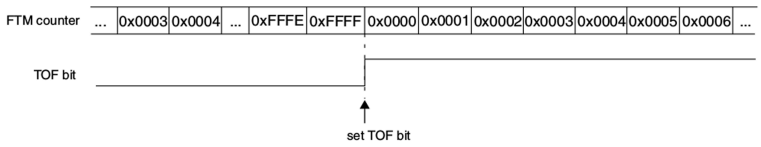


Figura 19: Free Running Counter

> ⚙️ **Registro de Filtro de Overflows (NUMTOF)**
>
> El FTM incluye el registro `NUMTOF[4:0]`, el cual especifica la cantidad de veces que el contador debe sufrir un overflow antes de que el hardware active efectivamente el bit de interrupción `TOF = 1`. Esto permite extender los intervalos de interrupción sin sobrecargar la CPU.

Modos de Operación y Configuración por Canal

Modo Compatibilidad TPM (`FTMEN = 0`) vs. Modo FTM Avanzado (`FTMEN = 1`)

El módulo FTM incluye dos modos globales de operación definidos en el registro `FTMx_MODE`:

- **FTMEN = 0 (TPM Legacy):** Modulo compatible con los timers TPM antiguos (arquitectura HCS08). Solo permite el uso del conjunto básico de registros.
- **FTMEN = 1 (FTM Advanced Functions):** Habilita todas las funciones avanzadas: Combine Mode, Deadtime Insertion, Fault Control, Quadrature Decoder, PWM Synchronization e Inversión de Salidas.

> ⚠️ **¡Protección de Escritura (WPDIS)!**
>
> Para escribir en el bit `FTMEN` o en registros avanzados, se debe desactivar primero la protección contra escritura poniendo en `'1'` el bit `WPDIS` (`Write Protection Disable`) en `FTMx_MODE`.

Modos de Trabajo Individuales y Combinados

Modo de Canal	Configuración de Bits	Descripción / Comportamiento

Input Capture (IC)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 00`
	`ELSnB:ELSnA = 01` (Ascendente)
	`ELSnB:ELSnA = 10` (Descendente)
	`ELSnB:ELSnA = 11` (Ambos flancos)
		Guarda el valor instantáneo del contador `FTM_CNT` en `FTMx_CnV` al detectar el flanco configurado en el pin. Genera interrupción `CHnF = 1`.

Output Compare (OC)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 01`
	`ELSnB:ELSnA = 01` (Toggle)
	`ELSnB:ELSnA = 10` (Clear)
	`ELSnB:ELSnA = 11` (Set)
		Compara `FTM_CNT` con `FTMx_CnV`. Al coincidir (`FTM_CNT == CnV`), conmuta la salida del pin automáticamente y activa `CHnF = 1`.

Edge-Aligned PWM (EPWM)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=0`
	`MSnB:MSnA = 1X`
	`ELSnB:ELSnA = 10` (High-true)
	`ELSnB:ELSnA = X1` (Low-true)
		PWM alineado al borde con conteo ascendente. La salida cambia al inicio del período y al coincidir con `CnV`.
		Ancho de pulso:
		$$\mathrm{CnV} - \mathrm{CNTIN}$$
		.

Center-Aligned PWM (CPWM)	`DECAPEN=0`, `COMBINE=0`, `CPWMS=1` `ELSnB:ELSnA = 10` (High-true)	PWM centrado con conteo Up-Down. **Ancho de pulso:**

Modo de Canal	Configuración de Bits	Descripción / Comportamiento
		$2 \times (\text{CnV} - \text{CNTIN})$. **Período:** $2 \times (\text{MOD} - \text{CNTIN})$.
Combine PWM Mode	`FTMEN=1`, `COMBINE=1`, `CPWMS=0` Requiere un par de canales (n par y $n+1$).	El canal n define el borde de encendido (`CnV`) y el canal $n+1$ el borde de apagado (`C(n+1)V`). **Ancho de pulso:** $\lvert \text{C}(n+1)\text{V} - \text{CnV} \rvert$.

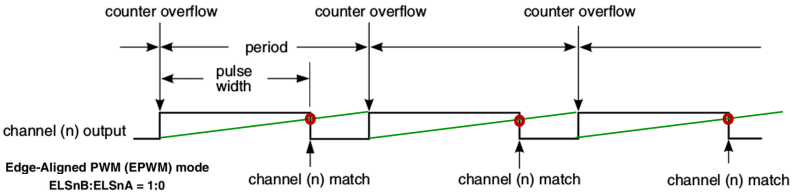


Figura 20: EPWM

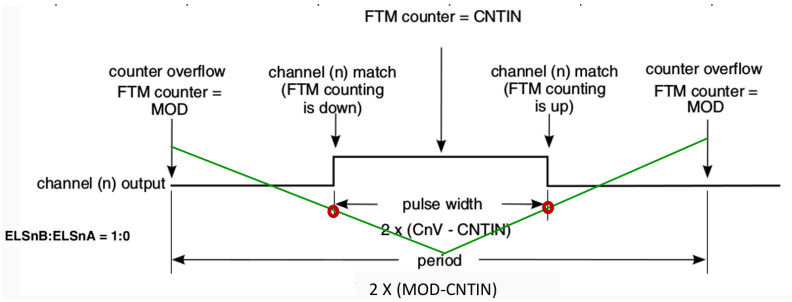


Figura 21: CPWM

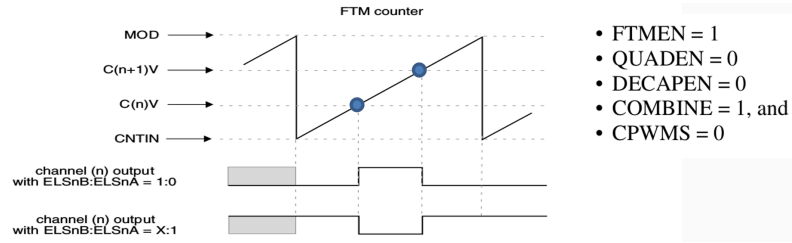


Figura 22: Combine PWM

Filtrado Digital en Captura de Entrada (Input Capture Filter)

El FTM incluye filtros digitales por hardware en los canales de entrada $0, 1, 2, 3$ para suprimir ruido o rebotes de señal.

El filtro exige que la señal permanezca estable durante una ventana temporal configurable en `FTMx_FILTER` mediante el campo `CHnFVAL[3:0]`:

$\text{Ventana de estabilidad} = 4 + 4 \times \text{CHnFVAL}[3{:}0]$ (en ciclos de clock del sistema)

Inserción de Tiempo Muerto (Deadtime Insertion)

En aplicaciones de electrónica de potencia (como el control de motores DC mediante puentes H o inversores trifásicos con MOSFETs/IGBTs), los dos conmutadores de una misma rama nunca deben conducir simultáneamente para evitar un cortocircuito directo (**shoot-through**) de la fuente de alimentación.

El FTM permite configurar salidas complementarias entre los canales n y $n+1$ (`COMP = 1` en `COMBINE`) e insertar de forma automática un tiempo muerto (**Deadtime**) a través del registro `FTMx_DEADTIME`:

⚙️ Registro FTMx_DEADTIME

- `DTPS[1:0]:` Prescaler del reloj de tiempo muerto (divisores $1, 4, 16$).
- `DTVAL[5:0]:` Número de ciclos de reloj del prescaler asignados como retardo de conmutación.

Control de Falla por Hardware (Fault Control)

El FTM monitorea hasta 4 entradas físicas de falla (`FAULTIN0..3`) para proteger las etapas de potencia. Ante una condición de sobrecorriente o sobretemperatura detectada por hardware, el módulo deshabilita inmediatamente las salidas PWM y las fuerza a un estado seguro sin esperar intervención del software.

Fuentes de Interrupción y Vectores NVIC

Cada módulo FTM consolida sus múltiples eventos de interrupción en **un único vector de interrupción** en el controlador NVIC del microcontrolador:

Fuente de Interrupción	Flags de Habilitación	Registro / Ubicación
Timer Overflow Interrupt	`TOIE = 1` y `TOF = 1`	Registro `FTMx_SC`.
Channel Interrupt (n)	`CHnIE = 1` y `CHnF = 1`	Registro de canal `FTMx_CnSC`.
Fault Interrupt	`FAULTIE = 1` y `FAULTF = 1`	Registro `FTMx_MODE`.

⚙️ Vectores IRQ en el Vector Table de K64F

- `FTM0_IRQHandler` → Vector 42 en NVIC (`FTM0_IRQn`).
- `FTM1_IRQHandler` → Vector 43 en NVIC (`FTM1_IRQn`).
- `FTM2_IRQHandler` → Vector 44 en NVIC (`FTM2_IRQn`).
- `FTM3_IRQHandler` → Vector 45 en NVIC (`FTM3_IRQn`).

Clase 9

ADC: Conversor Analógico–Digital

Arquitectura y Señales de Entrada

El Kinetis K64F incorpora **dos módulos ADC** de 16 bits con arquitectura **SAR (Successive Approximation Register)**. Cada instancia admite:

- **Hasta 24 canales Single Ended** (`AD[y]`, donde `y` es el número de canal).
- **4 canales Diferenciales** (`DAD[x][y]`, con `x = P(+)` o `M(−)`).

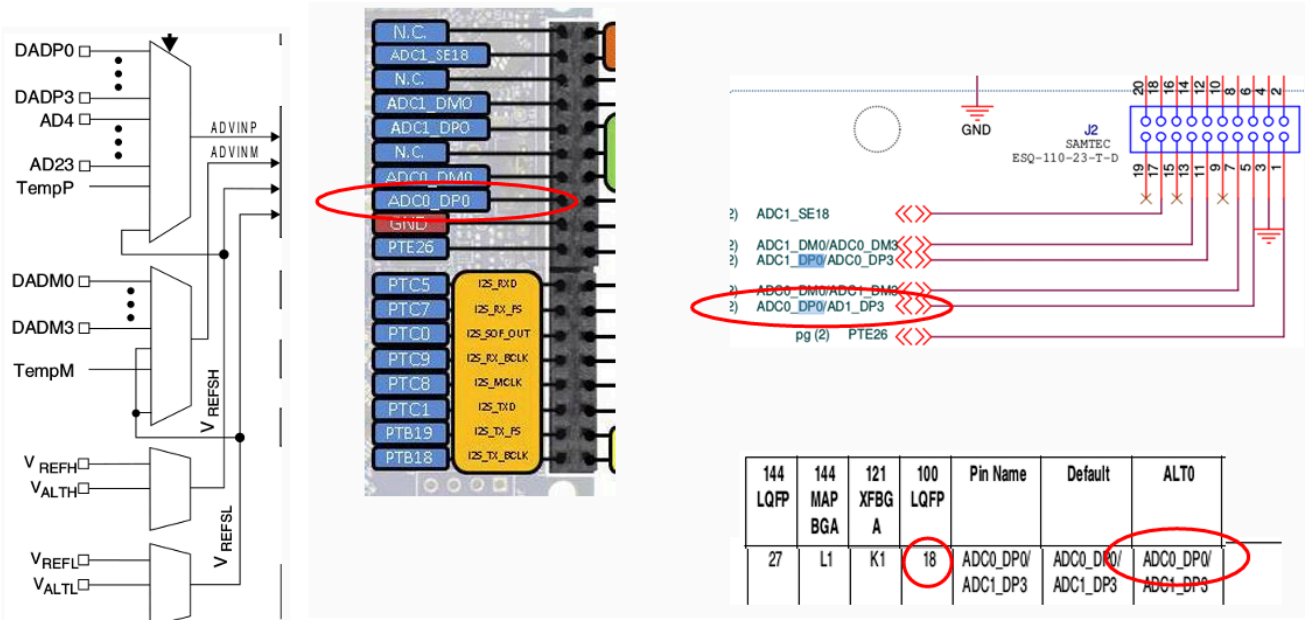


Figura 23: El canal a convertir se selecciona con el campo `ADCH[4:0]` del registro `ADCx_SC1`

> 💡 **Notación de Canales ADC**
>
> - **Diferencial:** `DAD[x][y]` donde `x = M(−)` o `P(+)`, `y = N° de canal`.
> - **Single Ended:** `AD[y]` — equivalente a `DADP[y]` cuando `DIFF = 0`.
>
> Al configurar `DIFF = 0`, el pin `DADP[y]` se reasigna como entrada Single Ended `AD[y]`.

Selección de Canal e Inicio de Conversión

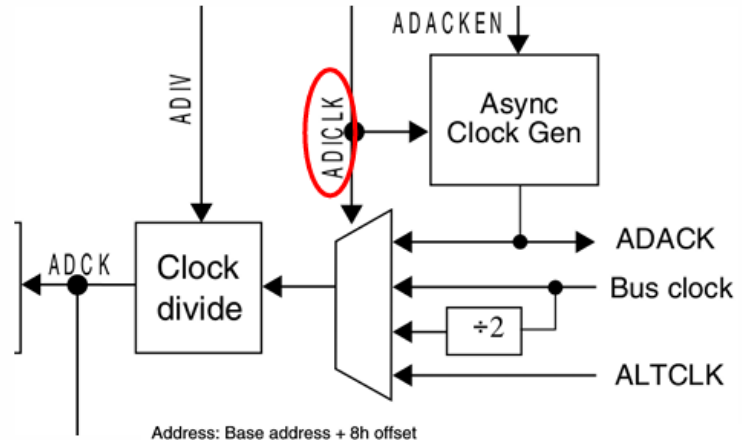
El canal a convertir y el modo de operación se configuran en el registro `ADCx_SC1[0]` mediante el campo `ADCH[4:0]`. Escribir en este registro **inicia la conversión**.

> ⚙️ **Registro ADCx_SC1 — Status and Control 1**
>
> - `ADCH[4:0]`: Selección del canal de entrada. Escribir en este campo inicia la conversión.
> - `DIFF`: `0` = modo Single Ended; `1` = modo Diferencial.
> - `AIEN`: `1` = habilita la interrupción al finalizar la conversión (`COCO = 1`).
> - `COCO` (read-only): **Conversion Complete Flag**. Se activa cuando la conversión finaliza. Se limpia al leer el registro `ADCx_Rn`.

```
// Inicio de conversión por software (polling o con interrupción)
void ADC_Start(ADC_t adc, ADCChannel_t channel, ADCMux_t mux)
```

```
{
    adc->CFG2 = (adc->CFG2 & ~ADC_CFG2_MUXSEL_MASK) | ADC_CFG2_MUXSEL(mux);
    adc->SC1[0] = ADC_SC1_AIEN(ADC_interrupt[x]) | ADC_SC1_ADCH(channel);
}
```

Reloj del ADC



El módulo ADC requiere una señal de reloj de entrada. Dicha señal se configura en `ADCx_CFG1` mediante dos campos:

Campo	Registro	Descripción
`ADICLK[1:0]`	`ADCx_CFG1[1:0]`	Selector de fuente de reloj: `00` = Bus Clock; `01` = Bus Clock/2; `10` = Alternate Clock (`ALTCLK`); `11` = ADACK (ADC Asynchronous Clock).
`ADIV[1:0]`	`ADCx_CFG1[6:5]`	Prescaler del reloj ADC. Factores de división: `00`=1; `01`=2; `10`=4; `11`=8.

```
ADC0->CFG1 = ADC_CFG1_ADICLK(0x00);  // Bus Clock como fuente
ADC0->CFG1 = ADC_CFG1_ADIV(0x00);    // Sin prescaler (dividir por 1)
```

Resolución del ADC

La resolución de conversión se configura en el campo `MODE[1:0]` del registro `ADCx_CFG1[3:2]`:

MODE[1:0]	DIFF = 0 (Single Ended)	DIFF = 1 (Diferencial, 2's Comp.)
`00`	8 bits	9 bits
`01`	12 bits	13 bits
`10`	10 bits	11 bits
`11`	16 bits	16 bits

Lectura del Resultado y Flag EOC

Una vez completada la conversión, el resultado se almacena en `ADCx_R[0]` y el flag `COCO` se pone en `1`.

```
// Verificar si la conversión finalizó (EOC - End Of Conversion)
bool ADC_IsReady(ADC_t adc)
{
    return adc->SC1[0] & ADC_SC1_COCO_MASK;
}
```

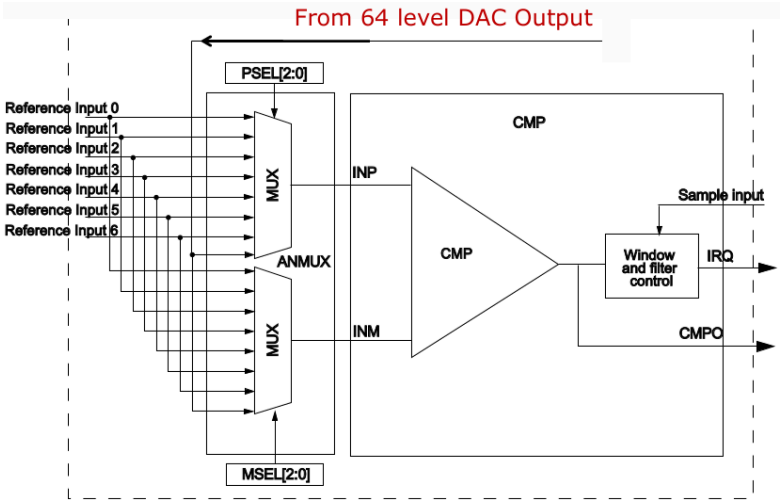
```
// Leer el resultado de la conversión (lectura limpia COCO)
ADCData_t ADC_getData(ADC_t adc)
{
    return adc->R[0];
}
```

> ⚠️ **Limpieza del Flag COCO**
>
> El flag `COCO` se limpia automáticamente al **leer** el registro `ADCx_Rn`. No es necesario limpiarlo de forma explícita.

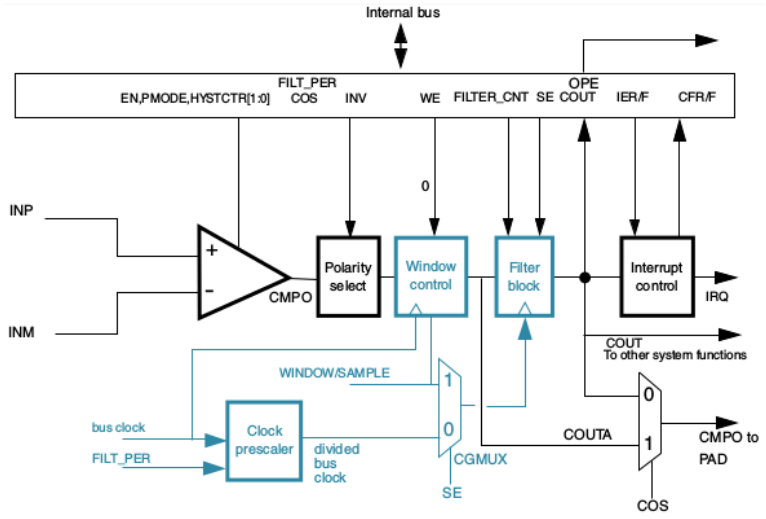
Comparador Analógico (CMP)

El K64F incorpora un **Comparador Analógico (CMP)** integrado que puede usarse en conjunto con el ADC. Sus características principales son:

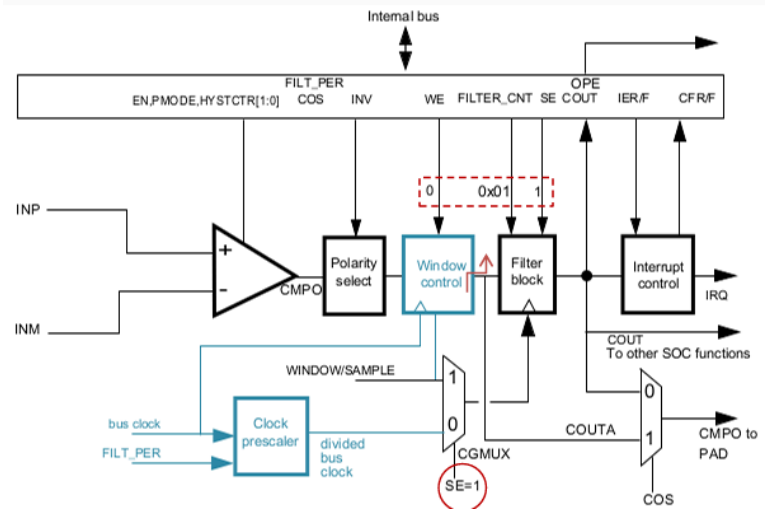
- **DAC interno de 64 niveles** para generar la tensión de referencia de comparación.



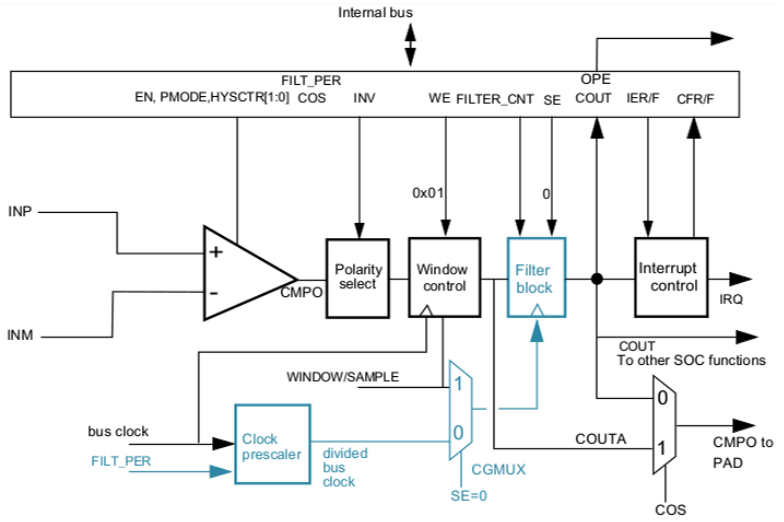
- **Control de histéresis** configurable por hardware para evitar oscilaciones en la salida.
- Múltiples modos de operación:
  - **Continuo:** La salida `COUT` refleja directamente la comparación en modo combinacional (sin reloj).

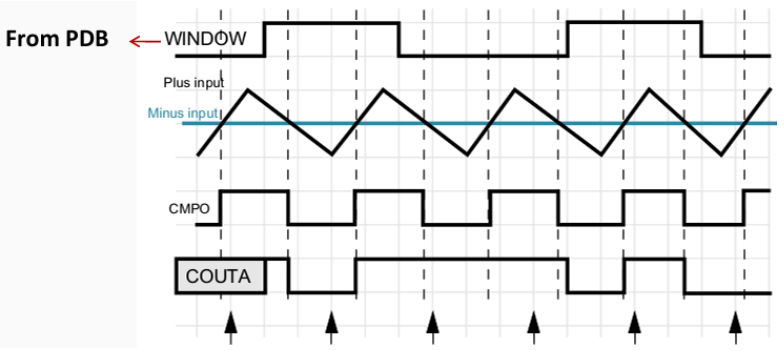


  - **Sampled Non-Filtered:** `COUTA` se muestrea en cada flanco ascendente del reloj del bloque de filtro.

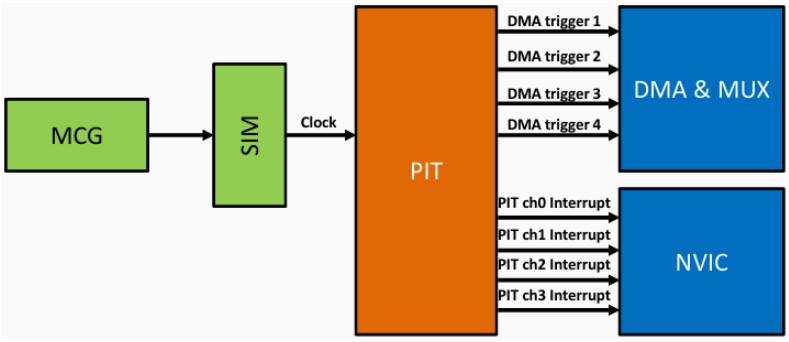


- **Windowed:** La salida analógica solo pasa a `COUTA` cuando la señal `WINDOW` está en alto (típicamente proveniente del PDB). El último valor queda latched cuando `WINDOW = 0`.





PIT: Periodic Interrupt Timer



Descripción General y Uso

El módulo **PIT (Periodic Interrupt Timer)** consiste en un conjunto de **4 timers de conteo descendente de 32 bits** cuyas funciones principales son:

- **Generar interrupciones periódicas** a intervalos configurables.
- **Proveer triggers periódicos** para los módulos DMA y ADC.

El PIT puede disparar transferencias DMA (a través del DMAMUX) y disparar conversiones ADC (a través de `SIM_SOPT7`), permitiendo adquisición y transferencia de datos completamente por hardware sin intervención de la CPU.

Clock y Habilitación

El **reloj del PIT es el Bus Clock** ($f_{\text{bus}} = 50$ MHz en la FRDM-K64F):

```
SIM->SCGC6 |= SIM_SCGC6_PIT_MASK;  // Habilitar clock gate del PIT
PIT->MCR = 0x00;                    // Encender el módulo PIT (MDIS = 0)
```

Registros Principales

Registro	Descripción
`PIT_MCR`	Module Control Register. `MDIS = 0`: módulo activo. `FRZ = 1`: detiene timers en modo debug.
`PIT_LDVALn`	Timer Load Value Register. Define el período: el timer cuenta desde `LDVALn` hasta `0` y genera la interrupción/trigger.
`PIT_CVALn`	Current Timer Value Register (solo lectura). Valor actual del contador descendente.
`PIT_TCTRLn`	Timer Control Register. `TEN = 1`: habilita el timer. `TIE = 1`: habilita la interrupción. `CHN = 1`: encadena el timer n al timer $n-1$ (Chain Mode).
`PIT_TFLGn`	Timer Flag Register. `TIF = 1`: indica que el timer expiró. Se limpia escribiendo `1` en este bit.

> 💡 **Cálculo del Período del PIT**
>
> Con $f_{\text{bus}} = 50$ MHz (período $t_{\text{bus}} = 20$ ns):
>
> $\text{LDVALn} = \frac{T_{\text{deseado}}}{t_{\text{bus}}} - 1$
>
> **Ejemplo:** Para $T = 5.12$ ms: $\text{LDVAL} = \frac{5.12 \times 10^{-3}}{20 \times 10^{-9}} - 1 = 256000 - 1$

Modo Chain (Timers Encadenados)

El **Chain Mode** permite extender el rango de temporización concatenando dos timers. El timer n solo decrementa cuando el timer $n-1$ llega a cero:

🛠️ Ejemplo: Timer de 1 Minuto con Chain Mode

Con $f_{\text{bus}} = 100\ \text{MHz}$ ($t_{\text{bus}} = 10\ \text{ns}$), para generar un período de 1 minuto ($60\ \text{s} = 6\ \text{s} \times 10$):

- **Timer 1:** $T = 6\ \text{s} \rightarrow \text{LDVAL1} = 6 \times 10^9\ \text{ns}\ /10\ \text{ns} - 1 = 6 \times 10^8 - 1$
- **Timer 2:** 10 ciclos del Timer 1 $\rightarrow \text{LDVAL2} = 10 - 1 = 9$ (con `CHN = 1`)

```
PIT->MCR = 0x00;
// Timer 2 (encadenado al Timer 1)
PIT->CHANNEL[2].LDVAL = 10 - 1;
PIT->CHANNEL[2].TCTRL = PIT_TCTRL_TIE_MASK | PIT_TCTRL_CHN_MASK | PIT_TCTRL_TEN_MASK;
// Timer 1 (fuente de clock para Timer 2)
PIT->CHANNEL[1].LDVAL = 600000000 - 1;
PIT->CHANNEL[1].TCTRL = PIT_TCTRL_TEN_MASK;
```

PIT como Trigger para DMA y ADC

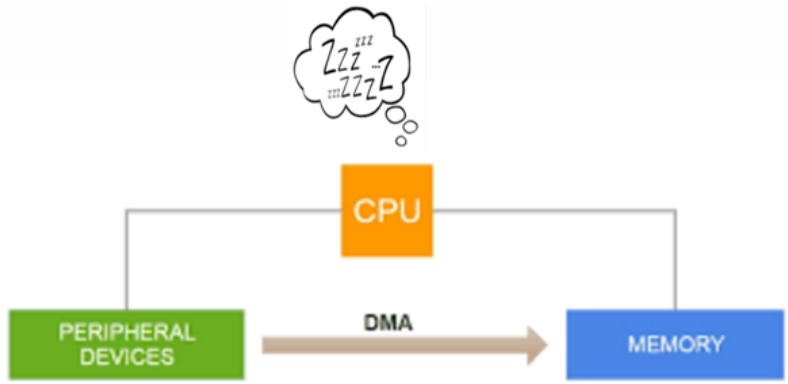
⚙️ PIT → DMA (DMAMUX)

Cada canal PIT (0..3) puede actuar como trigger periódico para el DMA MUX. Se configura en el DMAMUX activando `TRIG = 1` en `DMAMUX_CHCFGn`.

⚙️ PIT → ADC (SIM_SOPT7)

El PIT puede disparar conversiones ADC directamente. Se configura en `SIM_SOPT7` seleccionando el trigger y habilitando `ADCxALTTRGEN`.

DMA: Direct Memory Access



Concepto y Motivación

El **DMA (Direct Memory Access)** es un controlador de hardware que realiza transferencias de datos entre memoria y periféricos (o entre dos zonas de memoria) **sin intervención de la CPU**. Esto libera al procesador para ejecutar otras tareas mientras el DMA opera de forma autónoma sobre el bus del sistema.

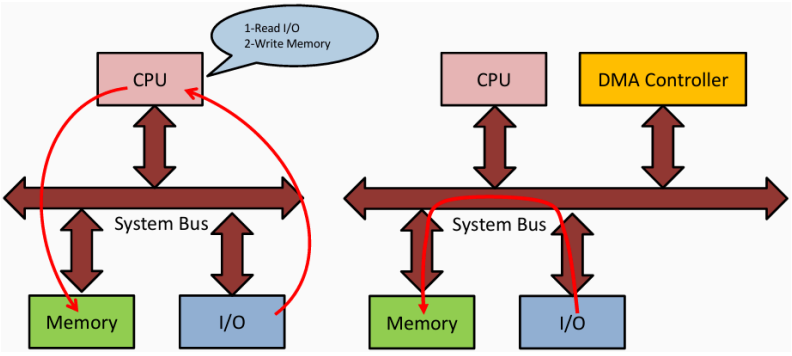


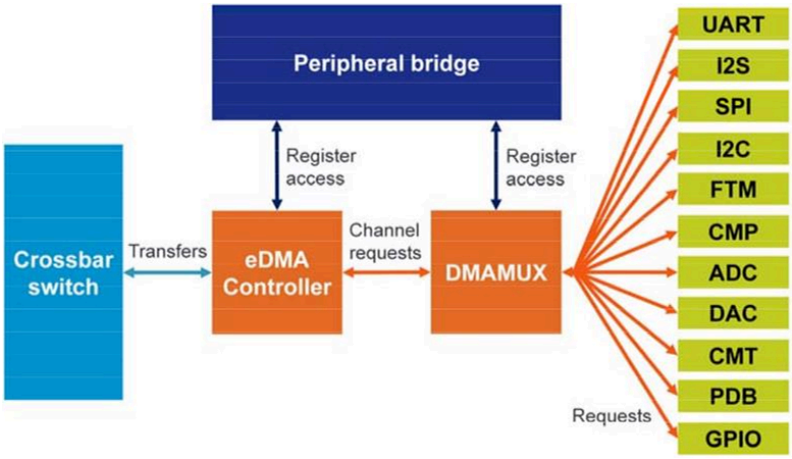
Figura 32: Programmed I/O Transfer vs **DMA Transfer**

El flujo de una transferencia DMA sigue estos pasos:

1. **Configuración:** La CPU programa los registros del controlador DMA (dirección origen, destino y cantidad de datos).
2. **Solicitud del Periférico:** El periférico genera un request de DMA cuando está listo para transferir datos.
3. **Arbitraje del bus:** El controlador DMA obtiene acceso al bus compartido para realizar la transferencia.
4. **Transferencia:** El DMA lee desde la dirección de origen y escribe en la dirección de destino; la CPU puede continuar ejecutando instrucciones cuando el bus está disponible.
5. **Fin de transferencia:** El contador del canal llega a cero. Si se habilitó la interrupción de fin de major loop, el DMA notifica a la CPU.

> ⚠️ **Diagrama conceptual de bus**
>
> Las señales `Hold Request`, `Hold Acknowledge` y `HI-Z` del diagrama ilustran un modelo general de DMA. En el K64F, el eDMA es un maestro del bus del sistema y su acceso se arbitra junto con el de la CPU; no se deben interpretar esas señales como pines externos del microcontrolador.

Arquitectura del eDMA en el K64F



El Kinetis K64F incorpora un módulo **eDMA (Enhanced DMA)** de **16 canales** con las siguientes características:

- **DMAMUX (DMA Multiplexer):** Rutea hasta **52 fuentes de DMA** (slots de periféricos) hacia los 16 canales del eDMA.

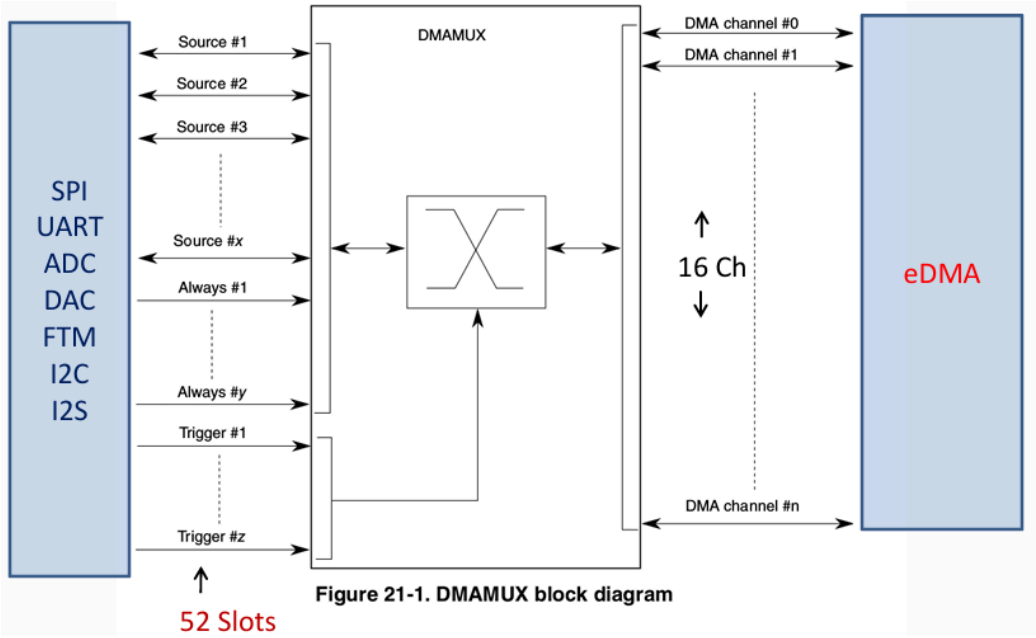


Figure 21-1. DMAMUX block diagram

- **TCD Memory:** Cada canal del eDMA tiene un **TCD (Transfer Control Descriptor)** de 32 bytes (8 palabras de 32 bits) que describe completamente la transferencia.

DMAMUX: Modos de Operación

El DMAMUX tiene tres modos configurables para cada canal mediante el registro `DMAMUX_CHCFGn`:

Modo	ENBL / TRIG	Comportamiento
Disabled	`ENBL=0`	Canal deshabilitado. Usar para reconfigurar el DMAMUX antes de cambiar la fuente.
Normal	`ENBL=1`, `TRIG=0`	La fuente del periférico (slot) se rutea directamente al canal DMA. La transferencia se inicia cuando el periférico genera un request.
Periodic Trigger	`ENBL=1`, `TRIG=1`	El request del periférico queda bloqueado (**gated**) hasta que el PIT genera un trigger. Solo los canales 0..3 soportan este modo.

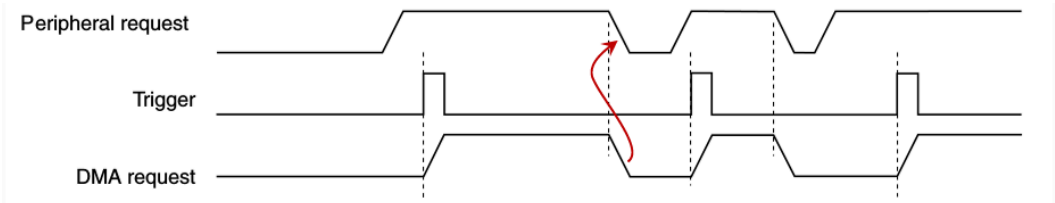


Figure 21-20. DMAMUX channel triggering: normal operation

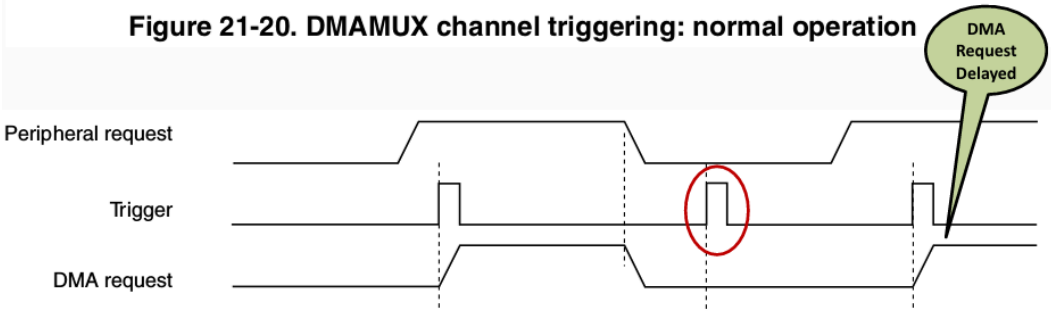


Figure 21-21. DMAMUX channel triggering: ignored trigger

⚙️ Registro DMAMUX_CHCFGn

- `ENBL`: Habilita el canal DMA.
- `TRIG`: Habilita el trigger periódico (modo Periodic Trigger con PIT).
- `SOURCE[5:0]`: Número de slot del periférico fuente (0..51). Ver tabla de slots en el Reference Manual del K64.

```
// Canal 0, fuente = SPI (slot 16), con trigger periódico del PIT
DMAMUX->CHCFG[0] = DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(16)
                 | DMAMUX_CHCFG_TRIG_MASK;

// Canal 0, fuente = PORTC (slot 51), modo normal (GPIO)
DMAMUX->CHCFG[0] = DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_SOURCE(51);

// Canal 0, fuente = Always-enabled (slot 63), con trigger del PIT
DMAMUX->CHCFG[0] = DMAMUX_CHCFG_ENBL_MASK | DMAMUX_CHCFG_TRIG_MASK
                 | DMAMUX_CHCFG_SOURCE(63);
```

Tipos de Fuentes DMA

Tipo de Fuente	Cant. de Canales	Características
Gated (Periféricos)	Slots 0..41	Throttled. El request queda gateado hasta que el periférico lo activa. Permite trigger periódico con PIT (canales 0..3).
Always-Enabled	10 slots (ej. slot 63)	Sin throttling. Transferencias rápidas: memoria→memoria, GPIO rápido, activación por software.

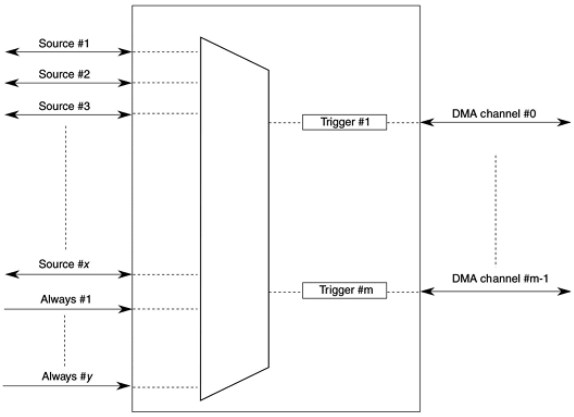


Figure 21-19. DMAMUX triggered channels

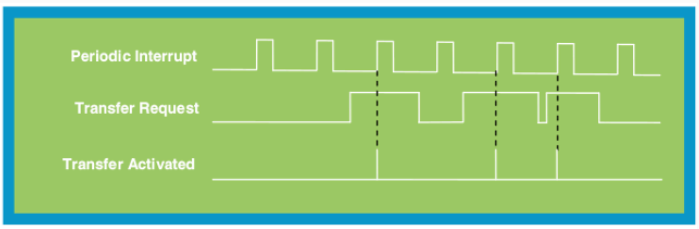


Figure 4. PIT gated transfer activation (Throttled)

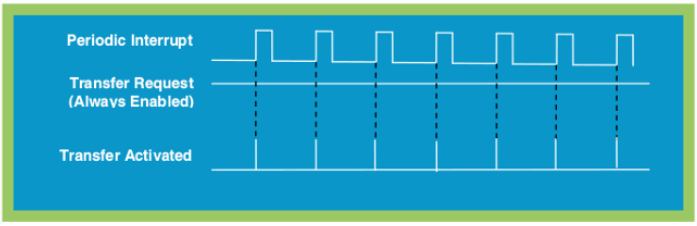


Figure 5. PIT only transfer activation

TCD: Transfer Control Descriptor

El TCD es la estructura que describe completamente cada transferencia DMA. Cada canal del eDMA tiene su propio TCD de 32 bytes en una memoria local:

```
typedef struct {
    uint32_t SADDR;          // Source Address (dirección de origen)
    uint16_t SOFF;           // Source Offset (offset por transferencia)
    uint16_t ATTR;           // Transfer Attributes (tamaño, SMOD, DMOD)
    uint32_t NBYTES_MLNO;    // Bytes por minor loop (por cada DMA request)
    uint32_t SLAST;          // Ajuste de SADDR al finalizar el major loop
    uint32_t DADDR;          // Destination Address (dirección de destino)
    uint16_t DOFF;           // Destination Offset (offset por transferencia)
    uint16_t CITER_ELINKNO;  // Contador actual del major loop
    uint32_t DLASTSGA;       // Ajuste de DADDR / puntero al siguiente TCD (S&G)
    uint16_t CSR;            // Control and Status (INTMAJOR, ESG, DREQ, etc.)
    uint16_t BITER_ELINKNO;  // Contador inicial del major loop (= CITER al inicio)
} TCD_t;
```

💡 Major Loop y Minor Loop

- **Minor Loop:** Una sola transacción de `NBYTES` bytes. Se ejecuta ante cada request del periférico.
- **Major Loop:** Conjunto de `CITER` minor loops. Al completarse el major loop, el DMA puede:
  - Generar una interrupción (`CSR[INTMAJOR] = 1`).
  - Ajustar `SADDR` sumando `SLAST` (para wrap-around de buffer circular).
  - Cargar un nuevo TCD en modo Scatter-Gather (`CSR[ESG] = 1`).

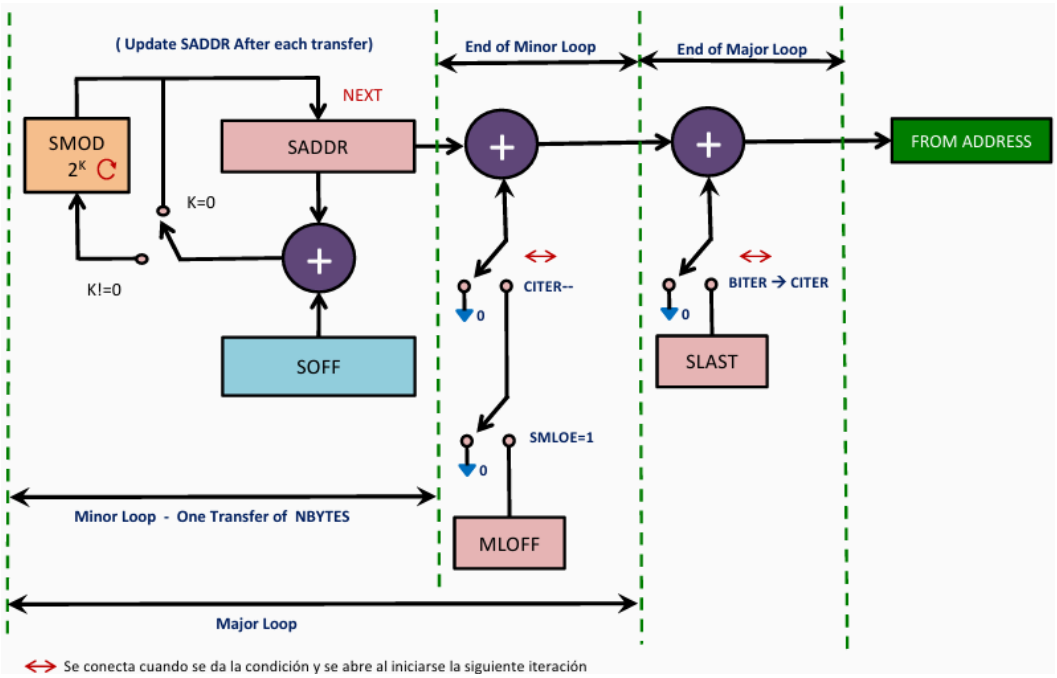


Figura 37: Source Address Generation Diagram

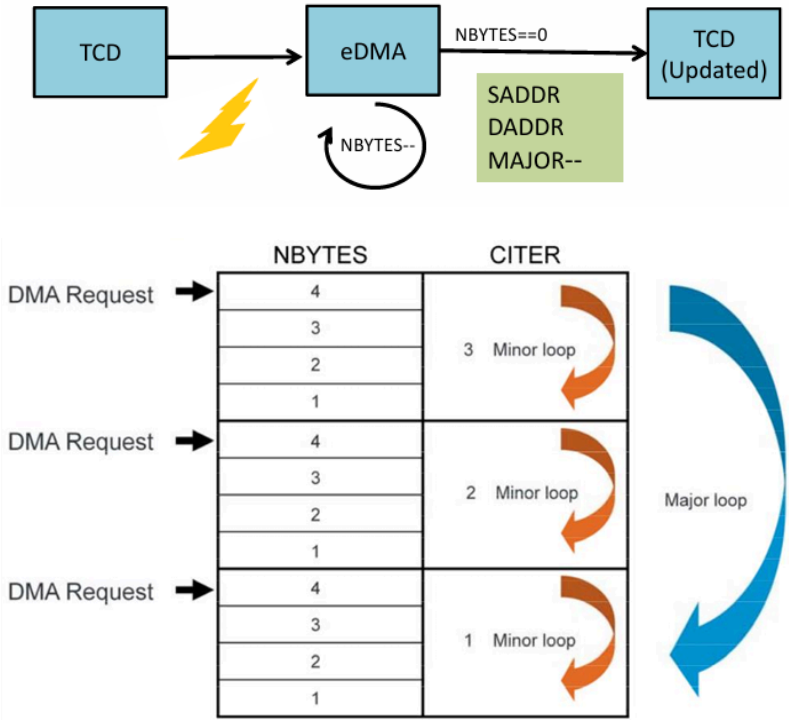


Figura 39: Transferencia de 4 bytes por minor loop

Campos Principales del TCD

Campo	Descripción
`SADDR`	Dirección de inicio de la fuente de datos.
`SOFF`	Offset que se suma a `SADDR` después de cada transferencia de origen. `0` = dirección fija (ej. registro de periférico).
`DADDR`	Dirección de inicio del destino de datos.
`DOFF`	Offset que se suma a `DADDR` después de cada transferencia de destino. `0` = dirección fija (ej. registro de periférico).

Campo	Descripción
`ATTR`	`SSIZE`/`DSIZE`: tamaño de dato (`0`=8bit, `1`=16bit, `2`=32bit). `SMOD`/`DMOD`: habilita buffer circular (ver más abajo).
`NBYTES`	Número de bytes transferidos por minor loop (por cada request del periférico).
`CITER`/`BITER`	Contador actual e inicial del major loop. **Deben inicializarse con el mismo valor.**
`SLAST`	Valor sumado a `SADDR` al finalizar el major loop. Negativo para wrap-around: `SLAST = -sizeof(buffer)`.
`DLASTSGA`	Ajuste de `DADDR` al finalizar el major loop. En modo Scatter-Gather apunta al próximo TCD.
`CSR`	Control: `INTMAJOR` = interrupción al fin del major loop; `ESG` = scatter-gather; `DREQ` = deshabilitar request al finalizar.

Generación de Direcciones y Tamaños de Transferencia

En cada transferencia, el eDMA accede a las direcciones actuales de origen (`SADDR`) y destino (`DADDR`). Después de cada elemento transferido, actualiza cada dirección con su offset:

$$\text{SADDR}_{\text{siguiente}} = \text{SADDR} + \text{SOFF} \quad \text{DADDR}_{\text{siguiente}} = \text{DADDR} + \text{DOFF}$$

Un offset igual a cero permite leer repetidamente un registro de periférico o escribir siempre en el mismo registro. Para recorrer una tabla en memoria, el offset suele ser igual al tamaño de cada elemento.

El campo `ATTR` selecciona independientemente el tamaño de los elementos de origen (`SSIZE`) y destino (`DSIZE`): `0` = 8 bits, `1` = 16 bits y `2` = 32 bits. Las direcciones y los tamaños deben ser compatibles con el periférico y con las restricciones de alineación de la memoria.

> **🛠️ Recorrer un arreglo de palabras**
>
> Para copiar palabras de 32 bits desde memoria a un registro fijo:
> - `SSIZE = DSIZE = 2` (32 bits).
> - `SOFF = 4` avanza al siguiente elemento del arreglo.
> - `DOFF = 0` conserva la dirección del registro de destino.
> - `NBYTES = 4` transfiere una palabra por cada servicio del canal.

Minor Loop y Major Loop

Un **minor loop** es la cantidad de bytes indicada por `NBYTES` para un servicio del canal. Puede incluir varias transferencias elementales; por ejemplo, `NBYTES = 64` con elementos de 32 bits mueve 16 elementos por servicio. El número de bytes debe ser compatible con los tamaños y alineaciones configurados.

`CITER` cuenta cuántos minor loops faltan para completar el **major loop**. El eDMA decrementa el contador al completar cada servicio. Al llegar a cero, puede aplicar los ajustes finales de dirección, generar una interrupción, activar otro canal y, si el canal sigue habilitado, recargar `CITER` desde `BITER`. Al configurar inicialmente el TCD, `CITER` y `BITER` deben tener el mismo valor; para un único servicio, ambos valen `1`.

El ancho disponible para el contador depende de `ELINK`: con linking desactivado se dispone de hasta 15 bits para el conteo; al habilitarlo, parte del campo se usa para seleccionar el canal enlazado y quedan 9 bits para el conteo.

NBYTES y Minor Loop Mapping

Con el mapeo de minor loop desactivado (`CR[EMLM] = 0`), se usa `NBYTES_MLNO`. Si se activa `EMLM`, se usan `NBYTES_MLOFFNO` (sin offset adicional) o `NBYTES_MLOFFYES` (con offset adicional habilitado).

En `NBYTES_MLOFFYES`, `SMLOE` y `DMLOE` determinan si el offset firmado `MLOFF` se suma a la dirección de origen o de destino al finalizar cada minor loop. Este ajuste es independiente de `SOFF` y `DOFF`, que se aplican después de cada elemento transferido.

> ⚠️ **Valor de NBYTES**
>
> No programes `NBYTES = 0` esperando una transferencia vacía: el eDMA interpreta ese valor como una transferencia de 2^{32} bytes. Configura un conteo válido para el modo de mapeo de minor loop seleccionado.

Ajustes al Final del Major Loop y Estado

- `SLAST` se suma a `SADDR` cuando termina el major loop. Un valor negativo puede volver al inicio de una tabla, por ejemplo `SLAST = -sizeof(buffer)`.
- Si `ESG = 0`, `DLASTSGA` se suma a `DADDR` al terminar el major loop.
- Si `ESG = 1`, `DLASTSGA` contiene la dirección del siguiente TCD en memoria. El descriptor debe cumplir la alineación requerida por el hardware.
- `CSR` controla, entre otras funciones, la interrupción de fin de major loop (`INTMAJOR`), Scatter-Gather (`ESG`) y la deshabilitación de solicitudes al finalizar (`DREQ`).

El flag `DONE` indica que el major loop terminó y `ACTIVE` que el canal está ejecutando una transferencia. Estos estados ayudan a comprobar el avance del canal durante la depuración.

Habilitación de Solicitudes del Canal

La selección de fuente en `DMAMUX_CHCFGn` y la habilitación de solicitudes del eDMA son pasos distintos. El canal debe tener su request habilitado en `DMA_ERQ` para aceptar solicitudes:

```
// Habilitar solicitudes del canal 0
DMA0->SERQ = DMA_SERQ_SERQ(0);

// Deshabilitar solicitudes del canal 0
DMA0->CERQ = DMA_CERQ_CERQ(0);
```

`SERQ` y `CERQ` modifican el bit correspondiente de `ERQ` sin requerir una operación de lectura-modificación-escritura sobre todo el registro. La configuración del TCD y del DMAMUX debe estar lista antes de habilitar la solicitud.

Buffer Circular con SMOD / DMOD

El campo `SMOD` (o `DMOD`) en `ATTR` define un **buffer circular por hardware** de tamaño $M = 2^K$ bytes. El hardware confina las actualizaciones de `SADDR` (o `DADDR`) a esa región. La dirección base debe estar alineada al tamaño del buffer: sus K bits menos significativos deben ser cero.

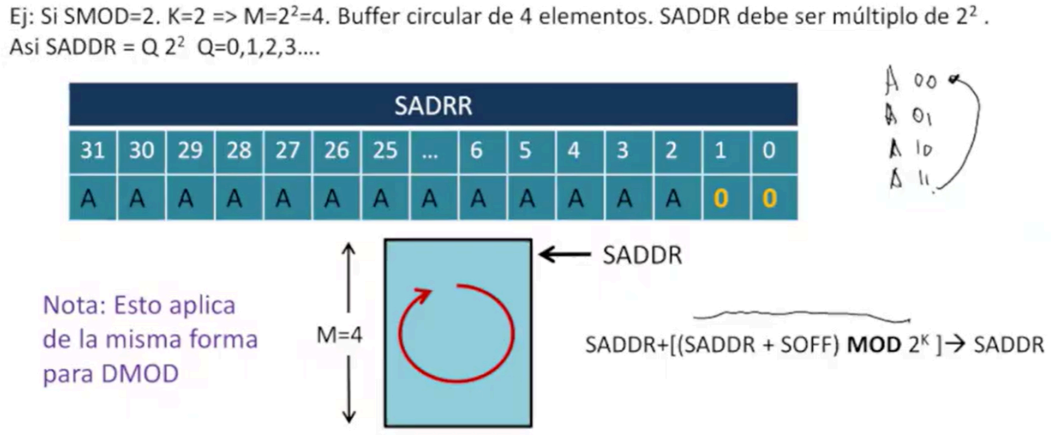


Figura 40: Ejemplo de buffer circular

```
// Buffer circular de 4 bytes: SMOD=2 → M = 2^2 = 4
DMA0->TCD[0].ATTR = DMA_ATTR_SSIZE(0) | DMA_ATTR_DSIZE(0) | DMA_ATTR_SMOD(2);
```

El tamaño definido por `SMOD`/`DMOD` siempre es potencia de dos. Para un buffer de tamaño arbitrario, por ejemplo 10 elementos, se puede usar Scatter-Gather con varios TCD que procesen segmentos del buffer.

Modos Avanzados del DMA

Scatter-Gather

El modo **Scatter-Gather** permite que un canal use varios TCD. Al finalizar el major loop, el hardware carga el siguiente TCD desde memoria principal sin intervención de la CPU. Los TCD pueden formar una lista circular si cada uno apunta al siguiente y el último vuelve al primero.

Esto se usa para implementar buffers de tamaño arbitrario (no potencia de 2) o esquemas de doble buffer (**ping-pong**):

```
// ESG = 1 en ambos descriptores; cada dirección debe apuntar a un TCD alineado.
TCD_A.DLASTSGA = (uint32_t)&TCD_B;
TCD_B.DLASTSGA = (uint32_t)&TCD_A;
```

Channel-to-Channel Linking

El **Channel Linking** activa otro canal al completar un minor loop (`ELINK`) o un major loop (`MAJORELINK`). El campo `LINKCH` o `MAJORLINKCH` selecciona el canal que se dispara. A diferencia de Scatter-Gather, no se carga un nuevo TCD ni se cambia la configuración del canal enlazado: se activa otro canal que ya fue configurado.

Característica	Scatter-Gather	Channel Linking
Qué ocurre	El canal carga otro TCD y cambia su configuración.	Se activa otro canal ya configurado.
Canales utilizados	Un mismo canal puede ejecutar varios TCD.	Participan varios canales independientes.
Memoria adicional	Requiere TCD en memoria alineados para los cambios.	No requiere TCD adicionales para enlazar canales.

Ejemplo: Buffer Circular de Tamaño No Potencia de Dos

Para representar un buffer de 10 elementos, el modo `SMOD`/`DMOD` no alcanza porque solo admite tamaños potencia de dos. Una alternativa es dividir la copia en dos major loops de 5 elementos:

1. El primer TCD copia los elementos `0..4` y parte de las direcciones base de origen y destino.
2. Al terminar el primer major loop, Scatter-Gather carga el segundo TCD, que copia los elementos `5..9` desde las direcciones desplazadas cinco elementos.
3. El segundo TCD puede apuntar de vuelta al primero. Al terminar cada mitad, una interrupción de major loop permite que la CPU procese el bloque y agregue nuevos datos al buffer de origen.

Así se implementa una secuencia circular de tamaño arbitrario usando TCDs encadenados y sin tener que mover los datos con la CPU.

Clase 10

Introducción a ARM Cortex-M4: Startup y Toolchain

> 💡 **Objetivo de la clase**
>
> El microcontrolador **no arranca directamente en `main()`**. Antes hay que dejar listo el **stack**, la **vector table** y la memoria (`.data` y `.bss`). En esta clase vemos (1) qué define la arquitectura ARM y cómo se organiza el Cortex-M4, (2) dónde vive cada variable en memoria, (3) qué hace la CPU después del reset y (4) cómo la **toolchain** genera el binario que se graba en la Flash.

CPU vs. microcontrolador

- El **ARM Cortex-M4** es una **CPU** (un núcleo).
- El **Kinetis MK64FN1M0VLL12** (NXP) es un **microcontrolador (MCU)**: la CPU más Flash, RAM, periféricos, reloj, lógica de reset, etc.

Esta distinción importa más adelante: **el reset lo maneja el MCU, no la CPU**.

Qué es ARM y qué define una arquitectura

- **ARM** (Advanced RISC Machine) es una **arquitectura de CPU**, no un fabricante.
- **Arm Holdings** es la empresa que **licencia** esa arquitectura a otras (NXP, ST, Microchip, Apple, etc.).
- La arquitectura es el **"contrato" entre hardware y software**: describe **qué** funcionalidad debe proveer el hardware al software, pero **no cómo** la implementa.

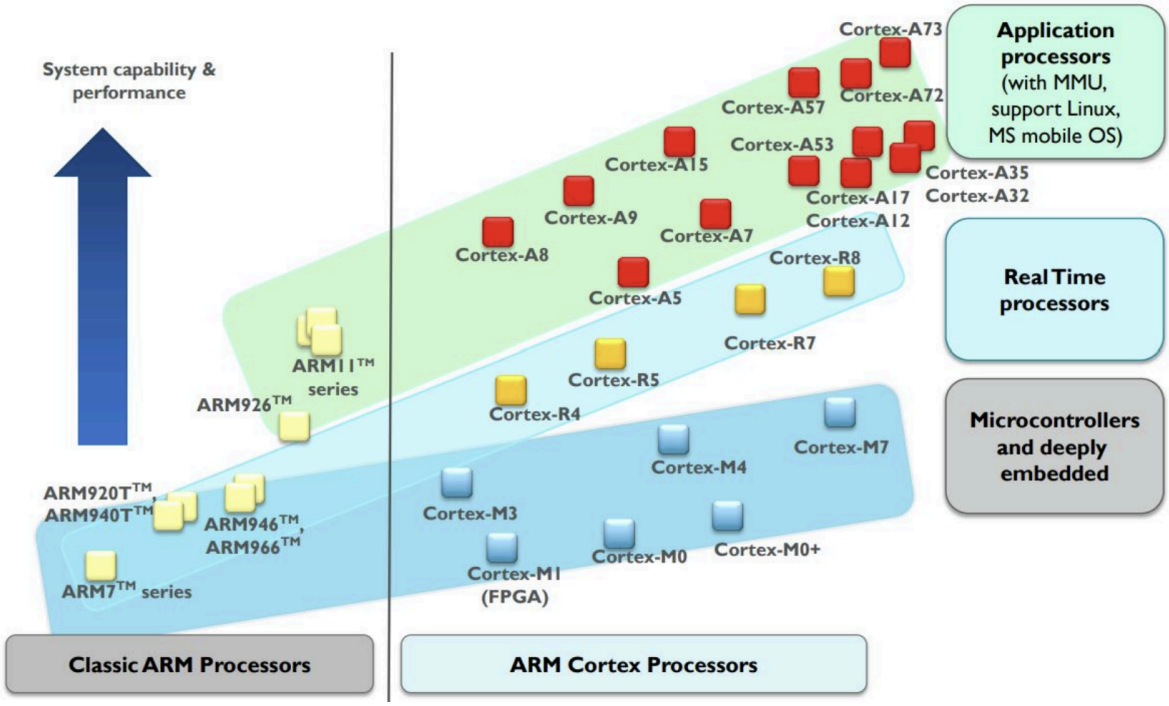
Elemento	Qué define
Instruction set (ISA)	Instrucciones disponibles, función de cada una (y qué excepciones puede disparar) y cómo se codifica en memoria.
Register set	Registros disponibles, su tamaño, su función y su comportamiento en el reset.
Exception model	Niveles de privilegio, tipos de excepciones y qué las dispara, y qué pasa al entrar o volver de una excepción.
Memory model	Orden de los accesos a memoria, comportamiento de las cachés y cuándo el software debe hacer mantenimiento explícito.
Debug, trace y profiling	Cómo se ponen y disparan los breakpoints, y qué información capturan las herramientas de trace y en qué formato.

Familias de procesadores ARM

ARM ofrece tres grandes perfiles, según el mercado al que apuntan:

	Cortex-A (Application)	**Cortex-R** (Real-time)	**Cortex-M** (Microcontroller)
Diseño	Alta frecuencia, pipeline largo, alta performance, soporte multimedia (NEON).	Pipeline corto a medio, determinístico (baja latencia de interrupción).	Pipeline corto, ultra bajo consumo, determinístico.
Features	MMU, caché, seguridad TrustZone, corre Linux/OS móviles.	MPU, caché, memoria TCM (Tightly Coupled Memory).	MPU, **NVIC** (controlador de interrupciones), WIC (wakeup).

	Cortex-A (Application)	**Cortex-R** (Real-time)	**Cortex-M** (Microcontroller)
Mercado	Smartphones, servidores, microprocesadores de alta gama.	Microcontroladores industriales, automotriz, discos rígidos, módems.	Sensores, MEMS, IoT, sistemas embebidos de bajo consumo.



Dentro de la familia Cortex-M:

Núcleo	**Característica principal**
M0 / M0+	Muy chicos, bajo costo y ultra bajo consumo. El M0+ agrega I/O de ciclo único y relocación de vector table.
M1	Diseñado para FPGAs (implementación con bloques de memoria de la FPGA). Mismo set de instrucciones que el M0.
M3	Más potente: set de instrucciones rico, divisor por hardware, MAC, debug y trace completos.
M4	Lo del M3 **más instrucciones DSP** (SIMD, MAC de ciclo único) y una **FPU opcional** de precisión simple (IEEE 754).
M7	Alto rendimiento: FPU de doble precisión opcional, cachés y TCM.

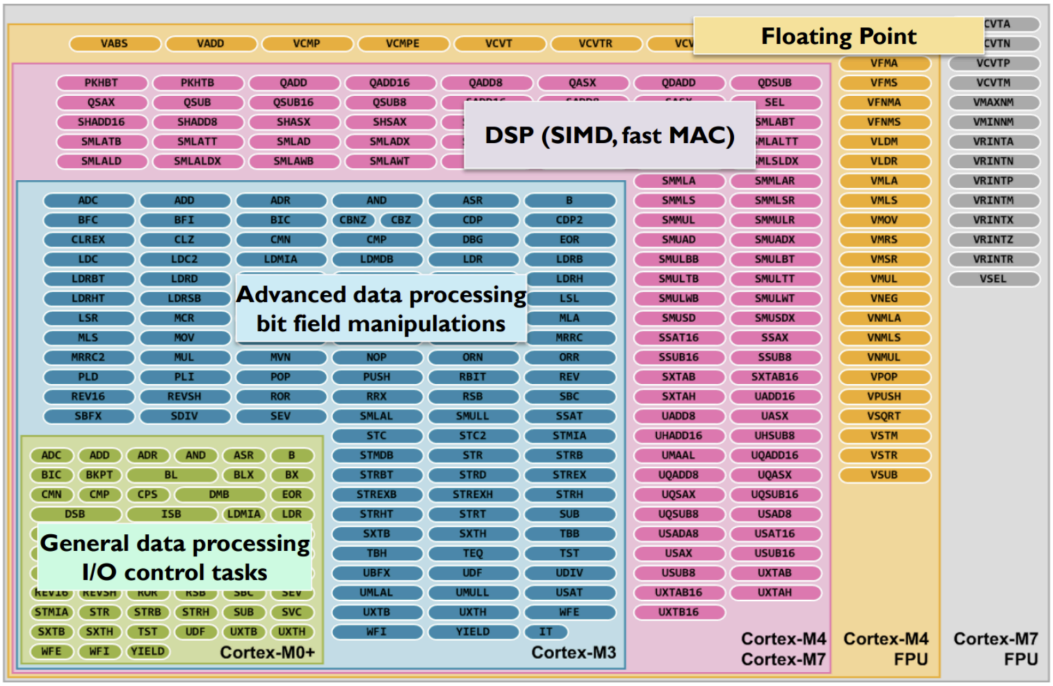


Figura 42: Soporte del Instruction Set en los distintos Cortex-M: M0/M0+, M3, M4/M7, DSP, FPU

Mapa de registros del Cortex-M4

El Cortex-M tiene registros de **32 bits**:

- **16 registros de propósito general** (`R0` a `R15`; tres de ellos tienen función especial: `SP`, `LR` y `PC`).
- **5 registros especiales**.
- **32 registros de punto flotante** (`S0`–`S31`, solo en variantes con FPU).

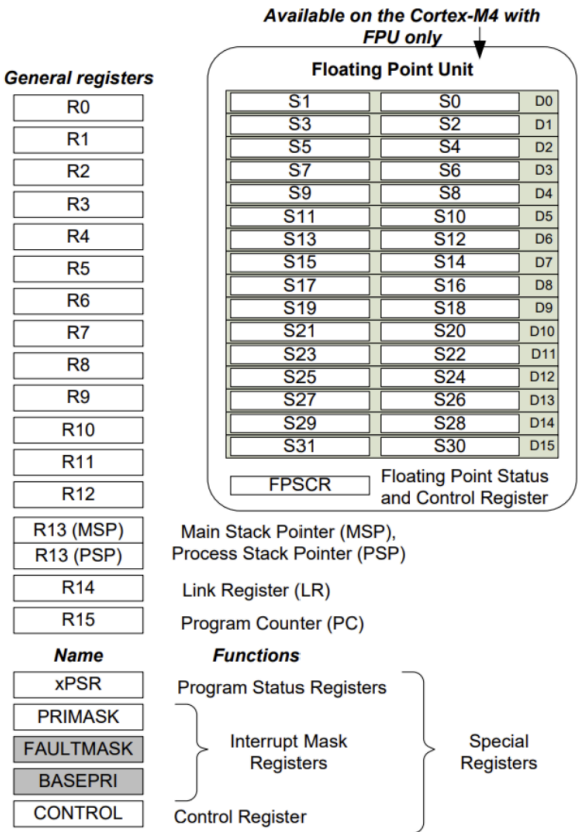


Figura 43: Mapa de registros del Cortex-M (R0–R15, registros especiales y registros de FPU

Registro	Función
`R0–R12`	Propósito general.
`R13` (`SP`)	**Stack Pointer**: puntero al stack. Existen dos copias, `MSP` (Main) y `PSP` (Process).
`R14` (`LR`)	**Link Register**: dirección de retorno de subrutina.
`R15` (`PC`)	**Program Counter**: instrucción que se está ejecutando.
`PSR`	Flags de la ALU, información de interrupciones y estado de ejecución.
`PRIMASK` (1 bit)	Bloquea **todas** las excepciones (incluidas las interrupciones), excepto `NMI` y `HardFault`.
`FAULTMASK` (1 bit)	Bloquea todas las excepciones, **incluso** `HardFault` (queda solo la `NMI`).
`BASEPRI` (8 bits)	Bloquea las excepciones con prioridad "mayor o igual a su valor" (numéricamente: valor de prioridad ≥ `BASEPRI`, o sea las menos urgentes).
`CONTROL`	Selecciona qué stack pointer se usa (`MSP` o `PSP`) y otras opciones de control.

> ⚠️ **Prioridades en ARM**
>
> En Cortex-M, **número menor = prioridad más alta**. Por eso `BASEPRI` enmascara las excepciones cuyo número de prioridad es **igual o mayor** al configurado (las menos urgentes) y deja pasar las más urgentes.

El Stack Pointer en acción

El stack se usa para guardar temporalmente registros y direcciones de retorno. Ejemplo típico: una subrutina que preserva `R4` y `R5` del llamador.

```
        ; Programa principal
        ; R4 = A, R5 = B
        BL   func1          ; salta a func1 y guarda en LR la dirección de retorno
        ; al volver: R4 = A, R5 = B

func1:
        PUSH {R4-R5}        ; guardar R4 y R5 en el stack
        ; ... tareas de la subrutina (puede usar R4 y R5 libremente) ...
        POP  {R4-R5}        ; restaurar R4 y R5
        BX   LR             ; retornar a main
```

- `PUSH` **decrementa** `SP` y escribe los registros: el stack **crece hacia direcciones bajas**.
- `POP` lee los valores y **incrementa** `SP`.

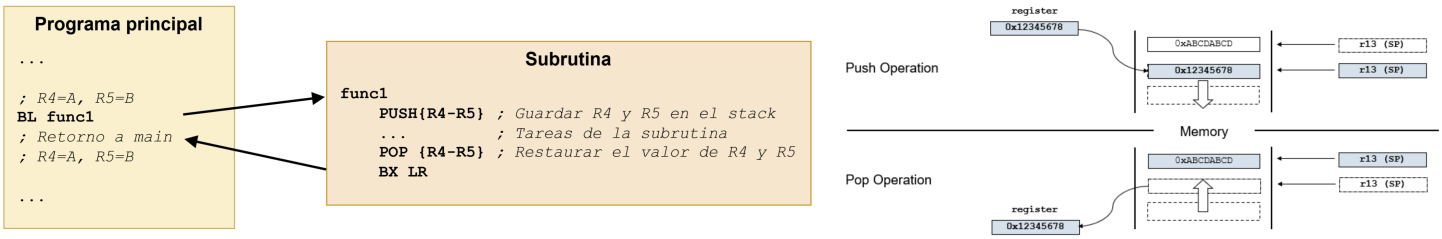


Figura 44: Operaciones Push y Pop sobre el stack

Arquitectura del Cortex-M4 y mapa de memoria

La arquitectura del Cortex-M4 es un **híbrido entre Harvard y Von Neumann**:

- Tiene **buses separados** para código y datos (como Harvard).

- Pero tiene un **único mapa de memoria unificado** (como Von Neumann).

Además, tiene componentes opcionales (FPU, MPU, ETM, watchpoints), el **NVIC** (que unifica el controlador de interrupciones entre los distintos fabricantes) y el **Debug Access Port**.

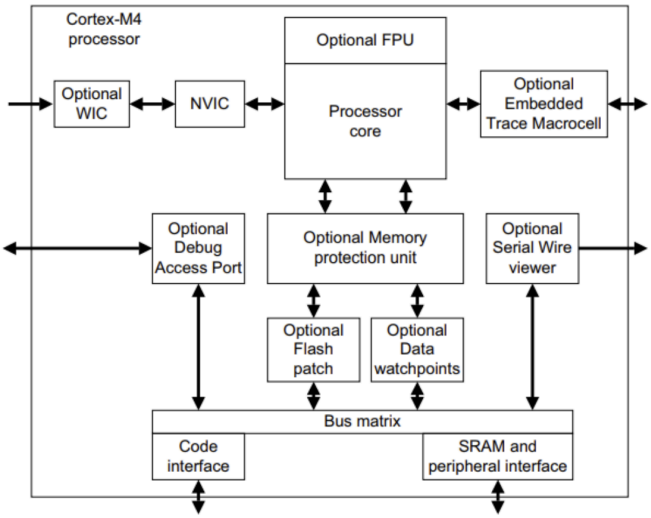


Figura 45: Diagrama de bloques de la implementación del Cortex-M4

> 💡 **Todos los Cortex-M4 comparten el mapa de memoria**
>
> El **mapa de memoria** (qué rango de direcciones corresponde a código, SRAM, periféricos, etc.) es **el mismo para todas las implementaciones** del Cortex-M4. Lo que **sí depende del fabricante/producto** es **qué hay realmente** en cada región: cuánta Flash y RAM tiene, qué periféricos incluye y en qué direcciones están sus registros.

Región	Tamaño	Rango de direcciones
Code (Flash)	0.5 GB	`0x0000_0000` – `0x1FFF_FFFF`
SRAM	0.5 GB	`0x2000_0000` – `0x3FFF_FFFF`
Peripheral	0.5 GB	`0x4000_0000` – `0x5FFF_FFFF`
External RAM	1 GB	`0x6000_0000` – `0x9FFF_FFFF`
External device	1 GB	`0xA000_0000` – `0xDFFF_FFFF`
Private peripheral bus	—	`0xE000_0000` – `0xE00F_FFFF`
System	—	`0xE010_0000` – `0xFFFF_FFFF`

Las regiones SRAM y Peripheral incluyen una **bit-band region** de 1 MB (con su **bit-band alias** de 32 MB) que permite acceder a bits individuales con una sola operación de lectura/escritura.

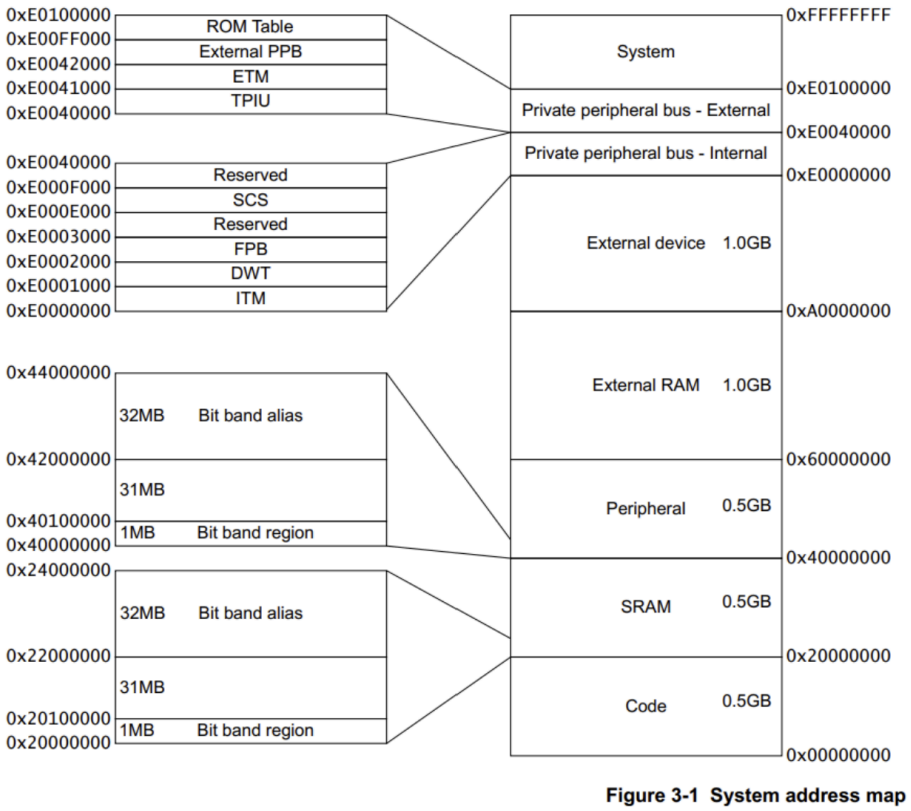


Figure 3-1 System address map

Figura 46: Mapa de memoria del sistema del Cortex-M4

¿Dónde se guarda cada variable?

Al compilar, el programa se divide en **secciones**. Cada tipo de variable o función cae en una sección distinta, y cada sección va a un tipo de memoria distinto.

Sección	Memoria	Qué contiene
`.text`	Flash	Código (instrucciones).
`.rodata`	Flash	Constantes (read-only).
`.data`	RAM (valores iniciales en Flash)	Variables globales/estáticas inicializadas con valor distinto de 0.
`.bss`	RAM	Variables globales/estáticas inicializadas en 0 (o no inicializadas).
heap	RAM	Memoria reservada dinámicamente (`malloc`).
stack	RAM	Variables locales y contexto de las llamadas a función.

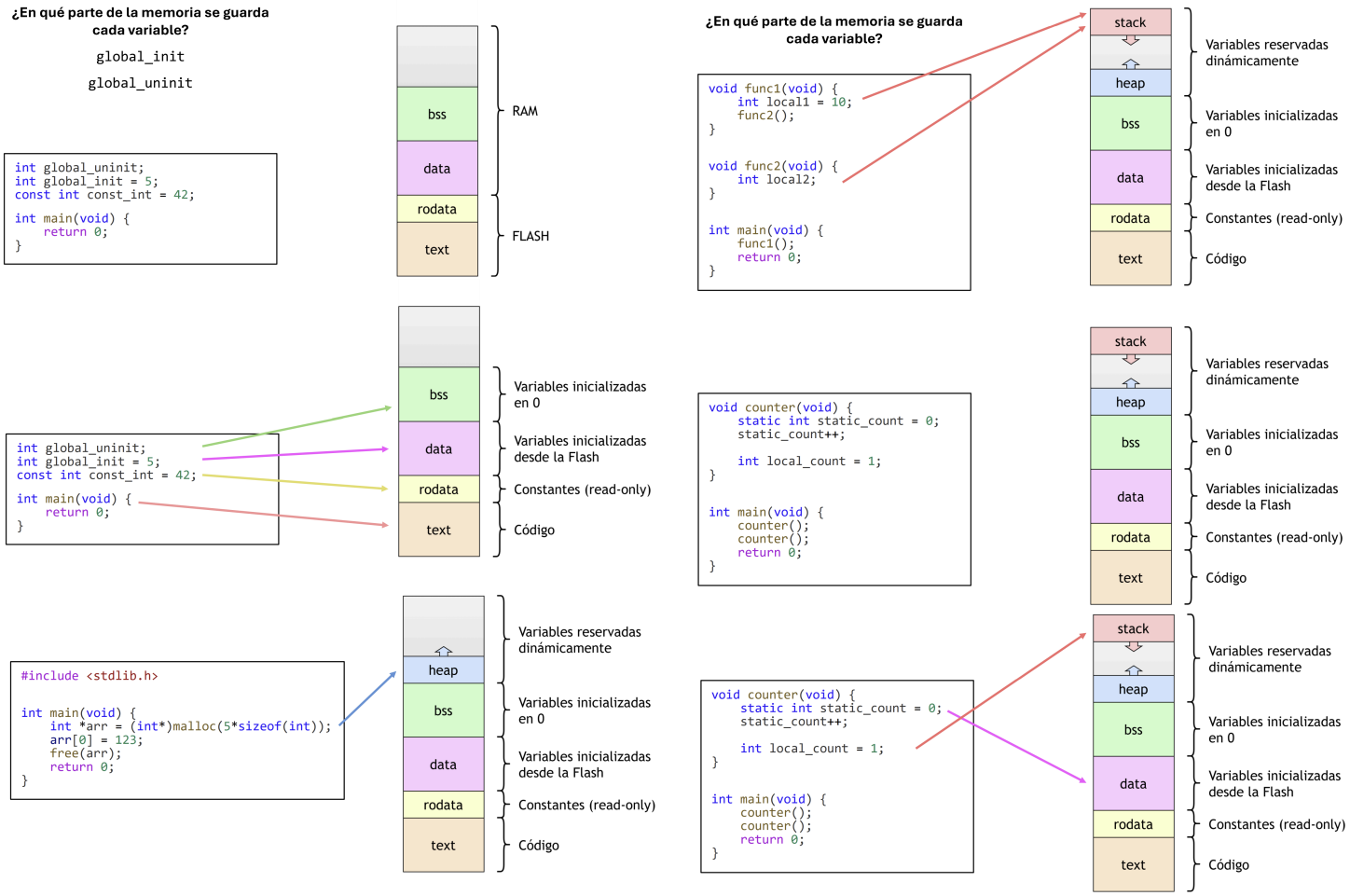


Figura 47: Layout de memoria: text, rodata, data, bss, heap y stack

Ejemplos de la clase

```c
const int const_int = 42;
const char * const const_string = "Hola";
int global_uninit;
int global_init = 5;

int main(void) {
    return 0;
}
```

Símbolo	Sección	Por qué
`main()`	`.text`	Es código.
`const_int`	`.rodata`	Constante: solo lectura, queda en Flash.
`const_string`	`.rodata`	Puntero constante a una cadena constante; ambos en Flash.
`global_init`	`.data`	Inicializada en 5: vive en RAM, pero su valor inicial se guarda en Flash.
`global_uninit`	`.bss`	Sin inicializar: vive en RAM y debe ponerse en 0.

Memoria dinámica (heap):

```c
#include <stdlib.h>

int main(void) {
    int *arr = (int*)malloc(5*sizeof(int));
    arr[0] = 123;
    free(arr);
    return 0;
}
```

- El bloque de 5 enteros que devuelve `malloc` está en el **heap**.
- El puntero `arr` en sí es una variable local y está en el **stack**.

Variables locales (stack):

```c
void func2(void) {
    int local2;
}

void func1(void) {
    int local1 = 10;
    func2();
}

int main(void) {
    func1();
    return 0;
}
```

`local1` y `local2` están en el **stack**. Cada llamada a función apila un nuevo marco, por eso `local2` queda "encima" de `local1` y desaparece al retornar `func2`.

Variables `static` locales:

```c
void counter(void) {
    static int static_count = 0;
    static_count++;

    int local_count = 1;
}

int main(void) {
    counter();
    counter();
    return 0;
}
```

- `local_count` está en el **stack**: se crea y se destruye en cada llamada.
- `static_count` **no está en el stack**: es una variable global con alcance local a la función, por eso **conserva su valor entre llamadas** (después de dos llamadas vale 2).

> ⚠️ **`static` inicializada en 0**
>
> En la diapositiva `static_count` se muestra en `.data`. En la práctica, GCC suele ubicar en `.bss` las variables inicializadas explícitamente en 0. En cualquier caso, lo importante es que **vive en RAM, fuera del stack**.

Booteo del ARM Cortex-M4

Supongamos el programa mínimo:

```
void main(void) {
    while (1) {
    }
}
```

La pregunta es: **¿cuál es la secuencia exacta desde que se energiza el micro hasta que ejecuta la primera instrucción de `main`? ¿Hay algo antes?**

Del power-on al reset

Al energizar el micro, la tensión empieza a subir. Mientras tanto, el MCU mantiene la CPU en reset. **El reset lo maneja la implementación del fabricante (el MCU), no la CPU Cortex-M4.**

- En el K64, el **Power-On Reset (POR)** se dispara cuando se aplica alimentación (o cuando cae por debajo de cierto umbral), y el circuito **LVD** (Low-Voltage Detect) mantiene el MCU en reset hasta que la tensión supera el umbral.
- Los valores concretos (tensión de POR, umbral de LVD) están en el **K64 Sub-Family Reference Manual** (sección Power-On Reset) y en el **Kinetis K64F Data Sheet** (requisitos de LVD y POR).

Qué hace la CPU después del reset

Después del reset (Power-On Reset u otro), la CPU:

1. Carga en `MSP` el **valor guardado en la dirección** `0x0000_0000`.
2. Carga en `LR` el valor `0xFFFF_FFFF`.
3. Carga en `PC` el **valor guardado en la dirección** `0x0000_0004`.

> ⚠️ **Es el contenido, no la dirección**
>
> `MSP` y `PC` se cargan con **lo que hay almacenado** en las direcciones 0 y 4, no con los números 0 y 4. La primera palabra de la memoria es el **valor inicial del stack pointer** y la segunda es la **dirección del handler de reset**.

Para bootear un Cortex-M4 hay que poner entonces en la Flash:

- Un valor en la dirección `0x0` para cargar en `MSP` (¿cuál?).
- Un valor en la dirección `0x4` para cargar en `PC` (¿cuál?).
- Los valores iniciales de las variables globales y las constantes.
- ¡Un programa!

Y además, **antes de ejecutar `main`**, el programa debe:

- cargar el valor **0** en las variables no inicializadas (`.bss`);
- **copiar de Flash a RAM** los valores iniciales de las variables inicializadas (`.data`).

> 💡 **Secuencia de arranque**
>
> **Reset** (hardware del MCU) → la CPU carga `MSP` y `PC` desde la **vector table** → se ejecuta el **reset handler** (startup) → se inicializan `.bss` y `.data` → se llama a `main()` → bucle infinito.

La toolchain ARM

Queremos generar, sin IDE ni templates (solo un editor de texto), el **contenido binario que va en la Flash**. Eso lo hace la **toolchain**.

ARM Toolchain: Flujo genérico

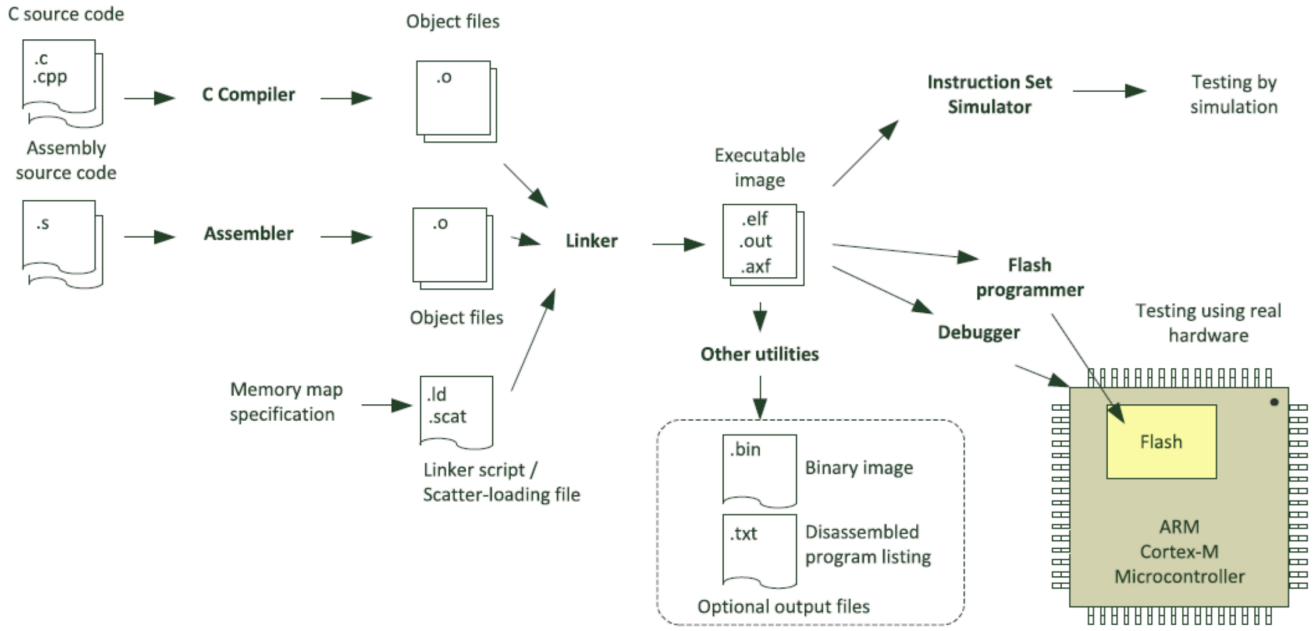


Figura 48: Flujo genérico de la toolchain ARM: compilador, ensamblador, linker, objcopy y flash programmer

- El **compilador** (C) y el **ensamblador** (assembly) generan **archivos objeto** (`.o`) de los codigos fuentes(c o assmebler).
- El **linker** combina los archivos objeto, según un **linker script** (la especificación del mapa de memoria), y genera la **imagen ejecutable** (`.elf`).
- Utilidades como `objcopy` convierten el ELF a formatos como `.bin` o `.hex`, y `objdump` genera un listado desensamblado.
- El **flash programmer** o el **debugger** graba esa imagen en la Flash del MCU real.

Nosotros usamos la toolchain **GNU** para ARM (`arm-none-eabi-*`, donde `none-eabi` indica programación **bare-metal**, sin sistema operativo).

Herramienta	Rol
`arm-none-eabi-gcc`	Compila C y ensambla assembly; también invoca al linker (`ld`) para generar el ELF.
`arm-none-eabi-ld`	Linker: asigna direcciones finales a las secciones y genera el ELF.
`arm-none-eabi-objdump`	Muestra el desensamblado y las tablas de símbolos y secciones.
`arm-none-eabi-objcopy`	Convierte el ELF a binario o hex para grabar en Flash.

Compilar: el archivo objeto

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O1 -c ej0.c -o ej0.o
$ arm-none-eabi-objdump -d ej0.o
```

```
ej0.o:     file format elf32-littlearm

Disassembly of section .text:

00000000 <main>:
   0:   e7fe            b.n    0 <main>
```

`b.n 0 <main>` es el `while (1)`: un salto de `main` a sí misma. La dirección es `0` porque un archivo objeto **todavía no fue linkeado**: las direcciones son relativas al inicio de su sección.

Linkear sin indicarle dónde va el código

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles ej0.o -o ej0.elf
warning: cannot find entry symbol _start; defaulting to 0000000000008000
$ arm-none-eabi-objdump -d ej0.elf
```

```
00008000 <main>:
    8000:       e7fe                b.n       8000 <main>
```

Ahora `main` quedó en `0x8000`: es la dirección por defecto del linker. **No es "incorrecto", pero no es lo que queremos**: el Cortex-M4 espera su vector table en la dirección 0 de la Flash.

> 💡 **¿A quién le decimos dónde poner el código?**
>
> **Al linker**, no al compilador. El compilador solo **asigna secciones** a cada pedazo del programa; el **linker asigna direcciones** a esas secciones, guiado por el **linker script** (`-T script.ld`).

Secciones y linker script

Programa de ejemplo:

```c
static int p = 100;
int q;
static const int r = 9;

void main(void) {
    while (p-- > r) {
        q++;
    }
}
```

```
$ arm-none-eabi-objdump -td ej1.elf
SYMBOL TABLE:
00008000 l  d .text      00000000 .text
00008030 l  d .rodata    00000000 .rodata
00018034 l  d .data      00000000 .data
00018038 l  d .bss       00000000 .bss
00018034 l  O .data      00000004 p
00008030 l  O .rodata    00000004 r
0001803c g    .bss       00000000 _bss_end__
00018038 g    .bss       00000000 __bss_start__
0001803c g    .bss       00000000 __bss_end__
00018038 g    .bss       00000000 __bss_start
00008000 g  F .text      00000030 main
0001803c g    .bss       00000000 __end__
00018038 g    .data      00000000 _edata
0001803c g    .bss       00000000 _end
00018038 g  O .bss       00000004 q
00018034 g    .data      00000000 __data_start
```

El compilador reparte cada elemento en su sección estándar:

Sección	Contenido	En el ejemplo
`.text`	Código.	`main`

Sección	Contenido	En el ejemplo
`.rodata`	Datos inicializados != 0, solo lectura (RO).	`r`
`.data`	Datos inicializados != 0, lectura/escritura (RW).	`p`
`.bss`	Datos inicializados en 0.	`q`

Sin linker script, el linker usa direcciones por defecto: `.text` en `0x8000`, `.rodata` en `0x8030`, y `.data` y `.bss` pegadas más arriba (`0x18034`, `0x18038`). **Ninguna de esas direcciones es útil para nuestro micro**, así que hace falta un script propio.

El linker script

```
MEMORY
{
    flash : ORIGIN = 0x00000000, LENGTH = 1024k
    ram   : ORIGIN = 0x20000000, LENGTH = 256K
}

SECTIONS
{
    .text :
    {
        . = ALIGN(4);
        *(.text*)
        *(.rodata*)
    } > flash

    .data :
    {
        . = ALIGN(4);
        _data_start = .;
        *(.data*)
        . = ALIGN(4);
        _data_end = .;
    } > ram AT > flash

    _data_init_values = LOADADDR(.data);

    .bss (NOLOAD) :
    {
        . = ALIGN(4);
        _bss_start = . ;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _bss_end = . ;
    } > ram
}
```

- `MEMORY` define las regiones físicas: `flash` (código y constantes) y `ram` (datos, stack).
- `.text` junta código y `.rodata` en Flash.
- `.data` se coloca en `ram` (**VMA**: dirección de ejecución) pero su contenido inicial se guarda en `flash` (**LMA**: dirección de carga), por eso `> ram AT > flash`. `LOADADDR(.data)` devuelve dónde quedaron esos valores iniciales en Flash.
- `.bss` es `NOLOAD`: ocupa RAM pero **no ocupa espacio en la Flash** (solo hay que ponerla en 0).
- El linker **define símbolos** (`_data_start`, `_data_end`, `_data_init_values`, `_bss_start`, `_bss_end`) que el código de inicio puede usar.

> ⚙️ **Nota sobre la RAM del K64**
>
> En los ejemplos de la clase la RAM arranca en `0x2000_0000` por simplicidad. En el K64 real, el reference manual reparte la SRAM a ambos lados de esa dirección (una parte arranca en `0x1FFF_0000`). Para un proyecto real hay que usar los valores del manual del MCU.

Compilar con el script

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles -T ej1.ld ej1.c -o ej1.elf
$ arm-none-eabi-objdump -td ej1.elf
SYMBOL TABLE:
00000000 l    d .text     00000000 .text
20000000 l    d .data     00000000 .data
20000004 l    d .bss      00000000 .bss
20000000 l    O .data     00000004 p
00000030 l    O .text     00000004 r
20000004 g    O .bss      00000004 q
00000000 g    F .text     00000030 main
20000000 g      .data     00000000 _data_start
20000004 g      .data     00000000 _data_end
20000004 g      .bss      00000000 _bss_start
20000008 g      .bss      00000000 _bss_end
00000034 g      *ABS*     00000000 _data_init_values
```

Ahora `main` está en `0x0`, `r` quedó en Flash (`0x30`), `p` y `q` en RAM (`0x2000_0000` y `0x2000_0004`), y los valores iniciales de `.data` están en Flash a partir de `0x34`.

El binario que va a la Flash

```
$ arm-none-eabi-objcopy -O binary ej1.elf ej1.bin
```

```
Offset    0   1   2   3   4   5   6   7   8   9   A   B   C   D   E   F
00000000  80  B4  00  AF  04  E0  08  4B  1B  68  01  33  06  4A  13  60
00000010  06  4B  1B  68  5A  1E  05  49  0A  60  09  22  93  42  F2  DC
00000020  00  BF  BD  46  80  BC  70  47  04  00  00  20  00  00  00  20
00000030  09  00  00  00  64  00  00  00
```

- Desde `0x00` hasta `0x2F`: el código de `main`.
- `0x30`: `09 00 00 00` = `r` = 9 (little endian).
- `0x34`: `64 00 00 00` = valor inicial de `p` = 100 (`0x64`), que alguien tendrá que **copiar a RAM**.

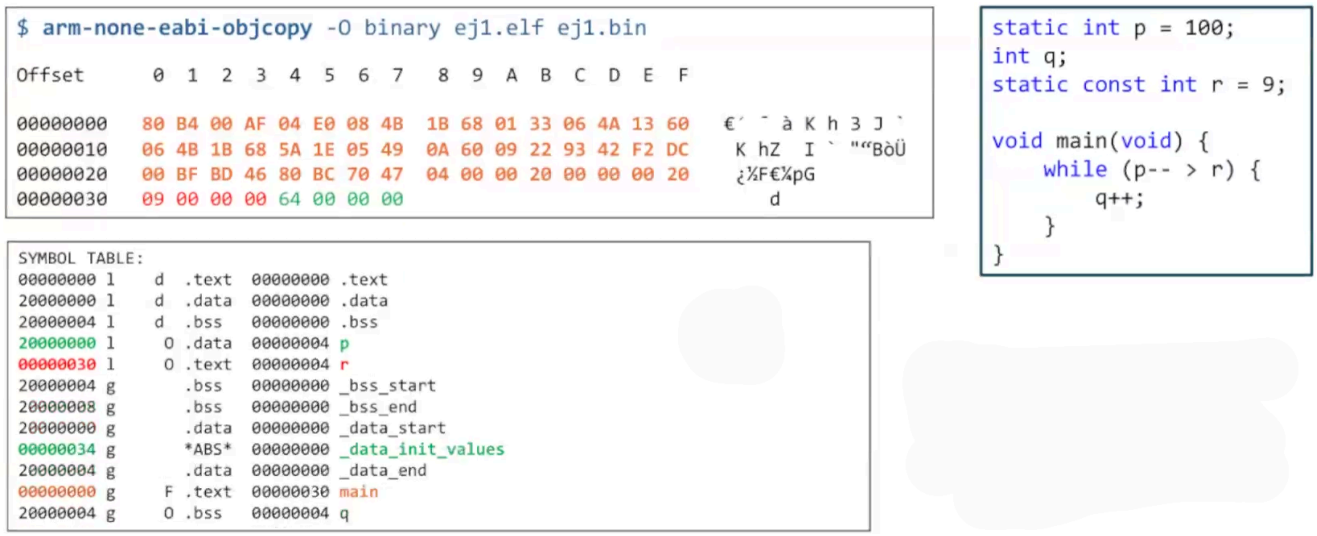


Figura 49: Ejemplo anterior.

> ⚠️ **¿Qué falta configurar?**
>
> Hasta acá el linker ordenó todo, pero **nadie usa los símbolos que generó**: no hay código que ponga `.bss` en 0 ni que copie `.data` a RAM. Y tampoco hay nada en las direcciones 0 y 4 que le diga al micro con qué `MSP` y `PC` arrancar. Falta el **código de startup**.

Código de startup

Inicialización pre-main (`init.c`)

```c
#include <string.h>

void init(void) {
    extern char _bss_start[];
    extern char _bss_end[];

    extern char _data_start[];
    extern char _data_end[];
    extern char _data_init_values[];

    // Poner .bss en 0
    memset(_bss_start, 0, (_bss_end - _bss_start));

    // Copiar los valores iniciales de .data desde Flash a RAM
    memcpy(_data_start, _data_init_values, (_data_end - _data_start));
}
```

Usa exactamente los símbolos que definió el linker script.

Vector table y reset handler (`startup.S`)

```
    .cpu cortex-m4
    .global _reset_vector

/* sección ".vectors", allocatable */
.section .vectors, "a"
    .word _sp_init_value      @ dirección 0x0: valor inicial de MSP
    .word _reset_vector       @ dirección 0x4: valor inicial de PC

.section .text
_reset_vector:
    bl init                   @ inicializar .bss y .data
    bl main                   @ ejecutar el programa de usuario
    b .                       @ si main retorna, quedarse acá
```

Las dos primeras palabras de la Flash son justo lo que la CPU lee después del reset: el stack inicial y la dirección del reset handler.

Linker script completo (`ej2.ld`)

```
MEMORY
{
    flash : org = 0, len = 1024k
    ram   : org = 0x20000000, len = 256K
}

ENTRY(_reset_vector)

SECTIONS
{
    .vectors :
```

```
    {
        *(.vectors*)
    } > flash

    ... (.text, .data y .bss como antes) ...

    _sp_init_value = org(ram) + len(ram);
}
```

- `.vectors` va **primero** en la Flash, para quedar en la dirección 0.
- `ENTRY(_reset_vector)` indica el punto de entrada.
- `_sp_init_value = org(ram) + len(ram)` coloca el stack al **final de la RAM**, y como el stack crece hacia abajo, tiene toda la RAM disponible.

Compilar y linkear todo junto

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c startup.S -o startup.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c init.c -o init.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c ej2.c -o ej2.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles -T ej2.ld startup.o init.o ej2.o -o ej2.elf
$ arm-none-eabi-objdump -td ej2.elf > ej2.dump.txt
$ arm-none-eabi-objcopy -O binary ej2.elf ej2.bin
```

Al inspeccionar el binario, el primer dato es `00 00 04 20`, que en little endian es `0x2004_0000`: el valor inicial de `MSP`, es decir **`0x2000_0000` + 256 KB** (el tope de la RAM). La segunda palabra es la dirección del reset handler, y a continuación está el código de `startup`, `init` y `main`.

> 🛠️ **Flujo completo de nuestro ejemplo**
>
> 1. Reset del MCU.
> 2. La CPU lee la dirección `0x0` y carga `MSP = 0x2004_0000`.
> 3. La CPU lee la dirección `0x4` y carga `PC = _reset_vector`.
> 4. `_reset_vector` llama a `init` (`.bss` en 0, `.data` copiada de Flash a RAM).
> 5. Se llama a `main`.
> 6. Si `main` retorna, `b .` deja la CPU en un bucle infinito.

Startup real del Kinetis (`startup_mk64f12.c`)

El SDK de Kinetis hace lo mismo pero en C y con la **vector table completa**.

Vector table. Es un arreglo de punteros a función que el linker script ubica al inicio de la Flash:

Dirección	Vector	Contenido
`0x0000_0000`	0	Valor inicial del stack pointer.
`0x0000_0004`	1	`ResetISR` (reset handler).
`0x0000_0008`	2	`NMI_Handler`.
`0x0000_000C`	3	`HardFault_Handler`.
`0x0000_0010`	4	`MemManage_Handler` (MPU fault).
`0x0000_0014`	5	`BusFault_Handler`.
`0x0000_0018`	6	`UsageFault_Handler`.
`0x0000_002C`	11	`SVC_Handler` (supervisor call).
`0x0000_0030`	12	`DebugMon_Handler`.

Dirección	Vector	Contenido
`0x0000_0038`	14	`PendSV_Handler`.
`0x0000_003C`	15	`SysTick_Handler`.
`0x0000_0040`	16 (IRQ 0)	Primera interrupción de periférico del chip (DMA canal 0).

Las posiciones reservadas se rellenan con `0`. Los primeros 16 vectores son **del núcleo ARM**; desde el vector 16 en adelante son **específicos del chip** (los periféricos del K64).

```
//*****************************************************************************
// The vector table.
// This relies on the linker script to place at correct location in memory.
//*****************************************************************************
extern void (* const g_pfnVectors[])(void);
extern void * __Vectors __attribute__ ((alias ("g_pfnVectors")));

__attribute__ ((used, section(".isr_vector")))
void (* const g_pfnVectors[])(void) = {
    // Core Level - CM4
    &_vStackTop,                    // The initial stack pointer
    ResetISR,                       // The reset handler
    NMI_Handler,                    // The NMI handler
    HardFault_Handler,              // The hard fault handler
    MemManage_Handler,              // The MPU fault handler
    BusFault_Handler,               // The bus fault handler
    UsageFault_Handler,             // The usage fault handler
    0,                              // Reserved
    0,                              // Reserved
    0,                              // Reserved
    0,                              // Reserved
    SVC_Handler,                    // SVCall handler
    DebugMon_Handler,               // Debug monitor handler
    0,                              // Reserved
    PendSV_Handler,                 // The PendSV handler
    SysTick_Handler,                // The SysTick handler

    // Chip Level - MK64F12
    DMA0_IRQHandler,                    // 16 : DMA Channel 0 Transfer Complete
    ...
```

Table 3-5. Interrupt vector assignments

Address	Vector	IRQ[1]	NVIC non-IPR register number [2]	NVIC IPR register number [3]	Source module	Source description
ARM Core System Handler Vectors						
0x0000_0000	0	–	–	–	ARM core	Initial Stack Pointer
0x0000_0004	1	–	–	–	ARM core	Initial Program Counter
0x0000_0008	2	–	–	–	ARM core	Non-maskable Interrupt (NMI)
0x0000_000C	3	–	–	–	ARM core	Hard Fault
0x0000_0010	4	–	–	–	ARM core	MemManage Fault
0x0000_0014	5	–	–	–	ARM core	Bus Fault
0x0000_0018	6	–	–	–	ARM core	Usage Fault
0x0000_001C	7	–	–	–	—	—
0x0000_0020	8	–	–	–	—	—
0x0000_0024	9	–	–	–	—	—
0x0000_0028	10	–	–	–	—	—
0x0000_002C	11	–	–	–	ARM core	Supervisor call (SVCall)
0x0000_0030	12	–	–	–	ARM core	Debug Monitor
0x0000_0034	13	–	–	–	—	—
0x0000_0038	14	–	–	–	ARM core	Pendable request for system service (PendableSrvReq)
0x0000_003C	15	–	–	–	ARM core	System tick timer (SysTick)
Non-Core Vectors						
0x0000_0040	16	0	0	0	DMA	DMA channel 0 transfer complete

Figura 50: Vector table del K64 (`g_pfnVectors`) y tabla de asignación de vectores de interrupción

Inicialización de RAM. Las funciones `data_init` y `bss_init` hacen lo mismo que nuestro `init.c`, pero copian/limpian palabra por palabra (de a 4 bytes):

```c
__attribute__ ((section(".after_vectors.init_data")))
void data_init(unsigned int romstart, unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int *pulSrc  = (unsigned int*) romstart;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = *pulSrc++;
}

__attribute__ ((section(".after_vectors.init_bss")))
void bss_init(unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = 0;
}
```

Reset handler. `ResetISR` recorre unas **tablas de secciones** (`__data_section_table` y `__bss_section_table`) para inicializar cada banco de RAM, y finalmente llama a `main()`. Se usan tablas porque algunos MCUs tienen **varias regiones de RAM**, y así el mismo `ResetISR` sirve para todos. Si `main` retorna, queda en un `while (1)`.

```c
//*****************************************************************************
// Functions to carry out the initialization of RW and BSS data sections. These
// are written as separate functions rather than being inlined within the
// ResetISR() function in order to cope with MCUs with multiple banks of
// memory.
//*****************************************************************************
__attribute__ ((section(".after_vectors.init_data")))
void data_init(unsigned int romstart, unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int *pulSrc = (unsigned int*) romstart;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = *pulSrc++;
}

__attribute__ ((section(".after_vectors.init_bss")))
void bss_init(unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = 0;
}
```

```c
//*****************************************************************************
// Reset entry point for your code.
// Sets up a simple runtime environment and initializes the C/C++
// library.
//*****************************************************************************
__attribute__ ((section(".after_vectors.reset")))
void ResetISR(void) {
    ...

    //
    // Copy the data sections from flash to SRAM.
    //
    unsigned int LoadAddr, ExeAddr, SectionLen;
    unsigned int *SectionTableAddr;

    // Load base address of Global Section Table
    SectionTableAddr = &__data_section_table;

    // Copy the data sections from flash to SRAM.
    while (SectionTableAddr < &__data_section_table_end) {
        LoadAddr = *SectionTableAddr++;
        ExeAddr = *SectionTableAddr++;
        SectionLen = *SectionTableAddr++;
        data_init(LoadAddr, ExeAddr, SectionLen);
    }
```

```c
    // At this point, SectionTableAddr = &__bss_section_table;
    // Zero fill the bss segment
    while (SectionTableAddr < &__bss_section_table_end) {
        ExeAddr = *SectionTableAddr++;
        SectionLen = *SectionTableAddr++;
        bss_init(ExeAddr, SectionLen);
    }

    ...

    main();

    //
    // main() shouldn't return, but if it does, we'll just enter an infinite loop
    //
    while (1) {
        ;
    }
}
```

Figura 51: `ResetISR`: copia de `.data` desde Flash a SRAM, puesta en cero de `.bss` y salto a `main`

Resumen final

💡 Ideas clave

1. **ARM define una arquitectura** (ISA, registros, excepciones, memoria, debug); el Cortex-M4 es una CPU, y el MCU (ej. Kinetis K64) es lo que la rodea.
2. **Todos los Cortex-M4 tienen el mismo mapa de memoria**; lo que cambia entre fabricantes es qué hay en cada región.
3. Cada variable va a una sección según su tipo: `.text`, `.rodata`, `.data`, `.bss`, **heap** y **stack**.
4. Tras el reset, la CPU carga `MSP` con el contenido de la dirección `0x0` y `PC` con el contenido de la dirección `0x4`: esa es la **vector table**.
5. El compilador asigna **secciones**; el **linker**, guiado por el **linker script**, asigna **direcciones**.
6. Antes de `main`, el startup debe poner `.bss` en 0 y copiar `.data` de Flash a RAM.

🛠️ Flujo típico

reset → vector table (MSP y PC) → startup → inicialización de `.bss` y `.data` → `main()` → bucle infinito